isye_6414

April 26, 2026

1 Railway Desert Analysis

This study is for railway access at the district level. The idea is to combine station, schedule, and train records into a single railway table, and then map that railway information to Census 2011 districts, then add demographic, Shrug, and economic context for regression model.

[1]:
```
import json
from pathlib import Path
import re
from difflib import get_close_matches

import pandas as pd
import numpy as np
import geopandas as gpd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.anova import anova_lm
from scipy.stats import jarque_bera, shapiro, skew, kurtosis
from sklearn.model_selection import train_test_split, KFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
import seaborn as sns

# Project visual palette: cold = low access, warm = middle, hot = high access.
cold_warm_hot_cmap = LinearSegmentedColormap.from_list(
    'cold_warm_hot',
    ['#2563eb', '#facc15', '#dc2626']
)
cold_color = '#2563eb'
warm_color = '#f59e0b'
hot_color = '#dc2626'
```

1.1 1. Data loading and preparation

[3]:
```
DATA_DIR = Path('../common_dataset/data')

# Railway
with open(DATA_DIR / 'rail' / 'stations.json', 'r', encoding='utf-8') as f:
    stations_geojson = json.load(f)
with open(DATA_DIR / 'rail' / 'trains.json', 'r', encoding='utf-8') as f:
    trains_geojson = json.load(f)

station_rows = []
for feat in stations_geojson['features']:
    props = feat.get('properties', {})
    geom = feat.get('geometry') or {}
    coords = geom.get('coordinates') if geom.get('type') == 'Point' else None
    lon = coords[0] if isinstance(coords, (list, tuple)) and len(coords) > 1␣
 ↪else np.nan
    lat = coords[1] if isinstance(coords, (list, tuple)) and len(coords) > 1␣
 ↪else np.nan
    station_rows.append({
        'code': props.get('code'),
        'name': props.get('name'),
        'state': props.get('state'),
        'zone': props.get('zone'),
        'address': props.get('address'),
        'lon': lon,
        'lat': lat,
    })

stations = pd.DataFrame(station_rows)
schedules = pd.read_json(DATA_DIR / 'rail' / 'schedules.json')

train_rows = []
for feat in trains_geojson['features']:
    props = feat.get('properties', {})
    train_rows.append({
        'train_number': str(props.get('number')),
        'train_name_master': props.get('name'),
        'train_type': props.get('type'),
        'train_zone': props.get('zone'),
        'from_station_code': props.get('from_station_code'),
        'from_station_name': props.get('from_station_name'),
        'to_station_code': props.get('to_station_code'),
        'to_station_name': props.get('to_station_name'),
        'distance': props.get('distance'),
        'duration_h': props.get('duration_h'),
        'duration_m': props.get('duration_m'),
```

```
        'first_ac': props.get('first_ac'),
        'second_ac': props.get('second_ac'),
        'third_ac': props.get('third_ac'),
        'sleeper': props.get('sleeper'),
        'chair_car': props.get('chair_car'),
        'first_class': props.get('first_class'),
    })

trains = pd.DataFrame(train_rows)

rail_combined = schedules.copy()
rail_combined['train_number'] = rail_combined['train_number'].astype(str)
rail_combined = rail_combined.merge(trains, on='train_number', how='left')
rail_combined = rail_combined.merge(
    stations.rename(columns={
        'code': 'station_code',
        'name': 'station_name_master',
        'state': 'station_state',
        'zone': 'station_zone',
        'address': 'station_address',
    }),
    on='station_code',
    how='left'
)

# Economy data
econ = pd.read_csv(DATA_DIR/'economy'/ 'Economy_Productivity_SD_India.csv')
sector_econ = pd.read_csv(DATA_DIR/'economy'/'District-Wise Sectoral Analysis␣
 ↪of Indian States.csv')

# Demographics data
demo = pd.read_csv(DATA_DIR/'demographics'/'demographics.csv',␣
 ↪encoding='iso-8859-1')
demo = demo.rename(columns={'ÿCountry': 'Country'})

# SHRuG Census 2011 district and subdistrict levels
shrug_dist = pd.read_csv(
    DATA_DIR/'shrug'/'secc_rural_pc11dist.csv',
    dtype={'pc11_state_id': str, 'pc11_district_id': str}
)
shrug_subdist = pd.read_csv(
    DATA_DIR/'shrug'/'secc_rural_pc11subdist.csv',
    dtype={'pc11_state_id': str, 'pc11_district_id': str, 'pc11_subdistrict_id':
 ↪ str}
)

# Census 2011 district polygons
```

```
districts = gpd.read_file(DATA_DIR/'demographics'/'Census_2011'/'2011_Dist.
 ↪shp')[
    ['DISTRICT', 'ST_NM', 'ST_CEN_CD', 'DT_CEN_CD', 'geometry']
]
```

1.2 2. Data inspection

[4]:
```
for name, d in [
    ('stations', stations),
    ('trains', trains),
    ('schedules', schedules),
    ('rail_combined', rail_combined),
    ('economy', econ),
    ('sector_economy', sector_econ),
    ('demographics', demo),
    ('shrug_district', shrug_dist),
    ('shrug_subdistrict', shrug_subdist),
    ('census_districts', districts),
]:
    print(f"\n{name}: shape={d.shape}")
    print(d.head(3))
```

```
stations: shape=(8990, 7)
      code     name      state  zone                        address        lon  \
0     BDHL   Badhal  Rajasthan   NWR  Kishangarh Renwal, Rajasthan  75.451645
1  XX-BECE  XX-BECE       None  None                           None        NaN
2  XX-BSPY  XX-BSPY       None  None                           None        NaN

         lat
0  27.252059
1        NaN
2        NaN

trains: shape=(5208, 17)
  train_number            train_name_master train_type train_zone  \
0        04601  Jammu Tawi Udhampur Special       DEMU         NR
1        04602       UDHAMPUR JAMMUTAWI DMU       DEMU         NR
2        04603             JAT UDAHMPUR DMU       DEMU         NR

  from_station_code from_station_name to_station_code to_station_name  \
0               JAT        JAMMU TAWI             UHP        UDHAMPUR
1               UHP          UDHAMPUR             JAT      JAMMU TAWI
2               JAT        JAMMU TAWI             UHP        UDHAMPUR

   distance  duration_h  duration_m  first_ac  second_ac  third_ac  sleeper  \
0      53.0         1.0        35.0         0          0         0        0
```

```
1      53.0         1.0        50.0         0          0         0        0
2      53.0         1.0        35.0         0          0         0        0

   chair_car  first_class
0          0            0
1          0            0
2          0            0

schedules: shape=(417080, 8)
  arrival  day                     train_name         station_name  \
0    None  1.0     Falaknuma Lingampalli MMTS  KACHEGUDA FALAKNUMA
1    None  1.0   Thrissur Guruvayur Passenger              THRISUR
2    None  1.0  Porbandar Muzaffarpur Express            PORBANDAR

  station_code      id train_number departure
0           FM  302214        47154  07:55:00
1          TCR  281458        56044  18:55:00
2          PBR  309335        19269  15:05:00

rail_combined: shape=(417080, 30)
  arrival  day                     train_name         station_name  \
0    None  1.0     Falaknuma Lingampalli MMTS  KACHEGUDA FALAKNUMA
1    None  1.0   Thrissur Guruvayur Passenger              THRISUR
2    None  1.0  Porbandar Muzaffarpur Express            PORBANDAR

  station_code      id train_number departure              train_name_master  \
0           FM  302214        47154  07:55:00     Falaknuma Lingampalli MMTS
1          TCR  281458        56044  18:55:00   Thrissur Guruvayur Passenger
2          PBR  309335        19269  15:05:00  Porbandar Muzaffarpur Express

  train_type  … third_ac sleeper chair_car first_class  station_name_master  \
0        Hyd  …        0       0         0           0  KACHEGUDA FALAKNUMA
1       Pass  …        0       0         0           0              THRISUR
2        Exp  …        1       1         0           0            PORBANDAR

   station_state  station_zone     station_address        lon        lat
0           None          None                None  78.474481  17.332197
1           None          None                None  76.207919  10.514790
2        Gujarat            WR  Porbandar, Gujarat  69.615506  21.643598

[3 rows x 30 columns]

economy: shape=(180, 9)
        City  Year  R&D Expenditure (% of GDP)  \
0  Ahmedabad  2019                        1.06
1  Ahmedabad  2020                        1.93
2  Ahmedabad  2021                        1.60
```

```
   Patents per 100,000 Inhabitants  Unemployment Rate (%)  \
0                              3.4                    5.6
1                              2.1                    6.6
2                              6.6                    5.8

   Youth Unemployment Rate (%)  SME Employment (%)  \
0                         11.8                38.5
1                         10.4                19.5
2                          8.9                16.7

   Tourism Sector Employment (%)  ICT Sector Employment (%)
0                            5.2                       20.7
1                            4.2                        6.8
2                            5.1                       13.4

sector_economy: shape=(2149, 20)
   Dist Code         Year  State Code    State Name Dist Name  \
0          1  2007 (2004)          14  Chhattisgarh      Durg
1          1  2008 (2004)          14  Chhattisgarh      Durg
2          1  2009 (2004)          14  Chhattisgarh      Durg

   PRIMARY SECTOR CONSTANT PRICES (in Millions Rs)  \
0                                            14348
1                                            12300
2                                            13876

   PRIMARY SECTOR CURRENT PRICES (in Million Rs)  \
0                                          18741
1                                          18452
2                                          22189

   PRIMARY SECTOR CONSTANT SHARES (Percent)  \
0                                     15.21
1                                     12.23
2                                     13.53

   PRIMARY SECTOR CURRENT SHARES (Percent)  \
0                                    16.06
1                                    13.37
2                                    15.65

   SECONDARY SECTOR CONSTANT PRICES (Millions in Rs)  \
0                                              45657
1                                              51149
2                                              49188

   SECONDARY SECTOR CURRENT PRICES (Millions in Rs)  \
0                                              56724
```

1	70733
2	66129

	SECONDARY SECTOR CONSTANT SHARES (Percent)
0	48.39
1	50.86
2	47.98

	SECONDARY SECTOR CURRENT SHARES (Percent)
0	48.61
1	51.26
2	46.63

	TERTIARY SECTOR CONSTANT PRICES (Millions in Rs)
0	34351
1	37116
2	39462

	TERTIARY SECTOR CURRENT PRICES (Millions in Rs)
0	41217
1	48798
2	53504

	TERTIARY SECTOR CONSTANT SHARES (Percent)
0	36.41
1	36.91
2	38.49

	TERTIARY SECTOR CURRENT SHARES (Percent)
0	35.32
1	35.36
2	37.73

	Total Constant Prices (in Millions Rs)
0	94356
1	100565
2	102526

	Total Current Prices (in Millions Rs)
0	116682
1	137984
2	141821

	Per Capita Current Prices (1000 in Rs)
0	37
1	43
2	44

```
demographics: shape=(784562, 9)
  Country         District  Ind_adm2_ID  point_order                State  \
0   India  Andaman Islands            0          326  Andaman and Nicobar
1   India  Andaman Islands            0          327  Andaman and Nicobar
2   India  Andaman Islands            0          328  Andaman and Nicobar

   sub_polygon_id  Latitude  Longitude  Number of Records
0              28   11.5957    92.5322                  1
1              28   11.5963    92.5325                  1
2              28   11.5969    92.5327                  1

shrug_district: shape=(615, 104)
  pc11_state_id pc11_district_id        secc_hh  sc_share  ed_prim_share  \
0            04              055    5149.000000  0.057891       0.606817
1            01              001  100277.662488  0.000581       0.485355
2            01              002   91528.932932  0.000249       0.408604

   ed_sec_share  land_own_share  nco2d_cultiv_share          tot_p  \
0      0.307322        0.041756            0.077871   21593.000000
1      0.196670        0.792136            0.258859  624902.512102
2      0.182761        0.764114            0.311738  632346.432870

            p_06  …  ag_inc_hh  veh_two_three_share  veh_four_share  \
0    2895.000000  …  23.061177             0.300307        0.015609
1  107363.061717  …  66.082989             0.038785        0.005232
2  122560.121083  …  70.192529             0.036859        0.007026

   veh_boat_share  inc_5k_plus_share  inc_10k_plus_share  _mean_p_miss  \
0        0.000068           0.454846            0.206254      0.000000
1        0.000136           0.262576            0.161046      0.037150
2        0.000053           0.285818            0.153573      0.005607

   _core_p_miss  _target_weight_share  _target_group_max_weight_share
0      0.000000                   1.0                             1.0
1      0.037034                   1.0                             1.0
2      0.005517                   1.0                             1.0

[3 rows x 104 columns]

shrug_subdistrict: shape=(5804, 105)
  pc11_state_id pc11_district_id pc11_subdistrict_id  secc_hh  sc_share  \
0            04              055               00277   5149.0  0.057891
1            12              252               01667    203.0  0.000000
2            18              324               02146    482.0  0.000113

   ed_prim_share  ed_sec_share  land_own_share  nco2d_cultiv_share    tot_p  \
0       0.606817      0.307322        0.041756            0.077871  21593.0
1       0.251111      0.086667        0.448276            0.938899    900.0
```

```
2       0.758695      0.182482        0.497925            0.324242   2329.0

   …   ag_inc_hh  veh_two_three_share  veh_four_share  veh_boat_share  \
0  …   23.061177             0.300307        0.015609        0.000068
1  …   71.714286             0.054187        0.000708        0.000708
2  …  118.000000             0.044193        0.006248        0.000000

   inc_5k_plus_share  inc_10k_plus_share  _mean_p_miss  _core_p_miss  \
0           0.454846            0.206254           0.0           0.0
1           0.103448            0.068966           0.0           0.0
2           0.136929            0.078838           0.0           0.0

   _target_weight_share  _target_group_max_weight_share
0                   1.0                             1.0
1                   1.0                             1.0
2                   1.0                             1.0

[3 rows x 105 columns]

census_districts: shape=(641, 5)
    DISTRICT           ST_NM  ST_CEN_CD  DT_CEN_CD  \
0   Adilabad  Andhra Pradesh         28          1
1       Agra   Uttar Pradesh          9         15
2  Ahmadabad         Gujarat         24          7

                                            geometry
0  POLYGON ((78.84972 19.7601, 78.85102 19.75945,…
1  POLYGON ((78.19803 27.4028, 78.19804 27.40278,…
2  MULTIPOLYGON (((72.03456 23.50527, 72.03337 23…
```

1.3 3. Standardization

I found that some district and state names were not consistent across multiple sources. So, I needed to create a standarized version so that district name could be consistent.

[4]:
```python
def rename_text(x):
    x = str(x).strip().lower()
    x = x.replace('&', 'and')
    x = re.sub(r'\s+', ' ', x)
    return x

state_map = {
    'delhi nct': 'nct of delhi',
    'delhi': 'nct of delhi',
    'orissa': 'odisha',
    'uttaranchal': 'uttarakhand',
    'andaman and nicobar': 'andaman and nicobar island',
    'dadra and nagar haveli': 'dadara and nagar havelli',
```

```
    'daman and diu': 'daman and diu',
    'jammu and kashmir': 'jammu and kashmir',
    'jammu & kashmir': 'jammu and kashmir',
    'arunachal pradesh': 'arunanchal pradesh',
}

district_map = {
    'ahmadabad': 'ahmedabad',
    'ahmadnagar': 'ahmednagar',
    'anantnag (kashmir south)': 'anantnag',
    'bangalore rural': 'bangalore rural',
    'bangalore urban': 'bangalore',
    'bengaluru': 'bangalore',
    'anugul': 'angul',
    'baramulla': 'baramula',
    'budgam': 'badgam',
    'boudh': 'bauda',
    'dhubri ': 'dhubri',
    'hardwar': 'haridwar',
    'hugli': 'hooghly',
    'panchmahal': 'panch mahals',
    'purbi singhbhum': 'purbi singhbhum',
    'pashchimi singhbhum': 'pashchimi singhbhum',
    'sant ravidas nagar': 'sant ravi das nagar(bhadohi)',
    'ysr': 'y.s.r.',
}

# rename census district keys
districts['district_norm'] = districts['DISTRICT'].apply(rename_text).
 ↪replace(district_map)
districts['state_norm'] = districts['ST_NM'].apply(rename_text).
 ↪replace(state_map)
districts['district_key'] = districts['district_norm'] + '|' +␣
 ↪districts['state_norm']
districts['state_census_code'] = pd.to_numeric(districts['ST_CEN_CD'],␣
 ↪errors='coerce').astype('Int64')
districts['district_census_code'] = pd.to_numeric(districts['DT_CEN_CD'],␣
 ↪errors='coerce').astype('Int64')

# rename shrug Census district identifiers
shrug_dist['state_census_code'] = pd.to_numeric(shrug_dist['pc11_state_id'],␣
 ↪errors='coerce').astype('Int64')
shrug_dist['district_census_code'] = pd.
 ↪to_numeric(shrug_dist['pc11_district_id'], errors='coerce').astype('Int64')
shrug_subdist['state_census_code'] = pd.
 ↪to_numeric(shrug_subdist['pc11_state_id'], errors='coerce').astype('Int64')
```

```
shrug_subdist['district_census_code'] = pd.
 ↪to_numeric(shrug_subdist['pc11_district_id'], errors='coerce').
 ↪astype('Int64')
shrug_subdist['subdistrict_census_code'] = pd.
 ↪to_numeric(shrug_subdist['pc11_subdistrict_id'], errors='coerce').
 ↪astype('Int64')
shrug_subdist['subdistrict_key'] = (
    shrug_subdist['pc11_state_id'].astype(str) + '|' +
    shrug_subdist['pc11_district_id'].astype(str) + '|' +
    shrug_subdist['pc11_subdistrict_id'].astype(str)
)

# rename railway
station_cols = ['code', 'name', 'state', 'zone', 'address', 'lon', 'lat']
stations = stations[station_cols].copy()
stations['state_norm'] = stations['state'].apply(rename_text).replace(state_map)
stations = stations.dropna(subset=['lon', 'lat'])

rail_combined['station_state_norm'] = rail_combined['station_state'].
 ↪apply(rename_text).replace(state_map)
rail_combined = rail_combined.dropna(subset=['station_code', 'lon', 'lat']).
 ↪copy()

# rename demographics
demo['district_norm'] = demo['District'].apply(rename_text).
 ↪replace(district_map)
demo['state_norm'] = demo['State'].apply(rename_text).replace(state_map)
demo['district_key_raw'] = demo['district_norm'] + '|' + demo['state_norm']

# rename economy city names
econ['city_norm'] = econ['City'].apply(rename_text)
latest_year = econ['Year'].max()
econ = econ[econ['Year'] == latest_year].copy()
econ = econ.fillna(econ.mean(numeric_only=True))

# rename district-wise sectoral economy keys
sector_econ['district_norm'] = sector_econ['Dist Name'].apply(rename_text).
 ↪replace(district_map)
sector_econ['state_norm'] = sector_econ['State Name'].apply(rename_text).
 ↪replace(state_map)
sector_econ['district_key_raw'] = sector_econ['district_norm'] + '|' +␣
 ↪sector_econ['state_norm']
sector_econ['sector_year'] = sector_econ['Year'].astype(str).str.
 ↪extract(r'(\d{4})')[0].astype(float)

demo = demo.fillna(demo.mean(numeric_only=True))
```

1.4 4. Mapping

Use rail_combined with station longitude and latitude to assign each scheduled railway stop to a Census district.

[5]:
```
rail_stop_gdf = gpd.GeoDataFrame(
    rail_combined.copy(),
    geometry=gpd.points_from_xy(rail_combined['lon'], rail_combined['lat']),
    crs='EPSG:4326',
)

# district assignment for stops that lie within the polygon boundary
rail_within = gpd.sjoin(
    rail_stop_gdf,
    districts[['district_key', 'district_norm',
    'state_norm', 'geometry']],
    how='left',
    predicate='within',
)

# nearest district for stops that are outside the polygon
outside = rail_within[rail_within['district_key'].isna()].copy()
inside = rail_within[rail_within['district_key'].notna()].copy()

if len(outside) > 0:
    drop_cols = [c for c in ['index_right', 'district_key',
    'district_norm', 'state_norm'] if c in outside.columns]
    outside_clean = outside.drop(columns=drop_cols)
    outside_projected = outside_clean.to_crs(epsg=3857)

    districts_projected = districts[['district_key', 'district_norm',
    'state_norm', 'geometry']].to_crs(epsg=3857)
    nearest_proj = gpd.sjoin_nearest(
        outside_projected,
        districts_projected,
        how='left',
        distance_col='nearest_dist_m',
    )
    nearest = nearest_proj.to_crs(epsg=4326)
    rail_district = pd.concat([inside, nearest], ignore_index=True)
else:
    rail_district = inside.copy()

rail_district = rail_district[rail_district['district_key'].notna()].copy()

state_col = 'state_norm_right' if 'state_norm_right' in rail_district.columns␣
 ↪else 'state_norm'
```

```
district_col = 'district_norm_right' if 'district_norm_right' in rail_district.
 ↪columns else 'district_norm'

rail_district = rail_district.rename(columns={
    district_col: 'district_norm_joined',
    state_col: 'state_norm_joined'
})

# So, the dataset is at station level and needs to be aggregated at the␣
 ↪district level
rail_aggreg = (
    rail_district.groupby(['district_key', 'district_norm_joined',␣
 ↪'state_norm_joined'])
    .agg(
        station_count=('station_code', 'nunique'),
        weekly_trains=('train_number', 'nunique'),
        stop_events=('id', 'count'),
        train_type_count=('train_type', 'nunique'),
        avg_train_distance=('distance', 'mean'),
        avg_duration_h=('duration_h', 'mean'),
        ac_train_share=('third_ac', 'mean'),
        sleeper_train_share=('sleeper', 'mean')
    )
    .reset_index()
    .rename(columns={
        'district_norm_joined': 'district_norm',
        'state_norm_joined': 'state_norm'
    })
)

print('Combined railway dataset shape:', rail_combined.shape)
print('Rail districts after assignment:', rail_aggreg.shape[0])
```

```
Combined railway dataset shape: (414420, 31)
Rail districts after assignment: 514
```

1.5 5. Combine datasets

`data_combi_1` is the district-level df. contains census districts and left joins the railway district summary, demographics, shrug indicators, and the existing city economy file.

`data_combi_2` is then created by left joining `data_combi_1` with all disticts and economic dataset.

[6]:
```
# Demographics aggregated
demo_agg = (
    demo.groupby(['district_key_raw', 'district_norm', 'state_norm'])
    .agg(
        geo_point_count=('Number of Records', 'sum'),
```

```
        avg_latitude=('Latitude', 'mean'),
        avg_longitude=('Longitude', 'mean')
    )
    .reset_index()
)

census_keys = set(districts['district_key'])
census_by_state = (
    districts[['state_norm', 'district_norm']]
    .drop_duplicates()
    .groupby('state_norm')['district_norm']
    .apply(list)
    .to_dict()
)

def map_to_census_key(district_norm, state_norm):
    key = f'{district_norm}|{state_norm}'
    if key in census_keys:
        return key
    candidates = census_by_state.get(state_norm, [])
    if not candidates:
        return np.nan
    best = get_close_matches(district_norm, candidates, n=1, cutoff=0.82)
    if best:
        return f'{best[0]}|{state_norm}'
    return np.nan

demo_agg['district_key'] = demo_agg.apply(
    lambda r: map_to_census_key(r['district_norm'], r['state_norm']),
    axis=1
)

demo_agg = (
    demo_agg[demo_agg['district_key'].notna()]
    .groupby('district_key')
    .agg(
        geo_point_count=('geo_point_count', 'sum'),
        avg_latitude=('avg_latitude', 'mean'),
        avg_longitude=('avg_longitude', 'mean')
    )
    .reset_index()
)

# shrug rural socioeconomic indicators at district level
shrug_feature_cols = [
    'secc_hh', 'tot_p', 'sc_share', 'st_share',
    'ed_prim_share', 'ed_sec_share', 'ed_grad_share',
```

```
    'land_own_share', 'inc_5k_plus_share', 'inc_10k_plus_share',
    'salary_any_share', 'salary_gov_share', 'salary_priv_share',
    'ag_inc_hh', 'veh_two_three_share', 'veh_four_share',
    'wall_mat_solid_share', 'roof_mat_solid_share',
    'num_room_mean', 'irr_share1', 'irr_share2'
]
shrug_feature_cols = [c for c in shrug_feature_cols if c in shrug_dist.columns]

shrug_agg = shrug_dist[['state_census_code', 'district_census_code'] +␣
 ↪shrug_feature_cols].copy()
for col in shrug_feature_cols:
    shrug_agg[col] = pd.to_numeric(shrug_agg[col], errors='coerce')

shrug_agg = (
    shrug_agg.groupby(['state_census_code', 'district_census_code'],␣
 ↪dropna=False)
    .mean(numeric_only=True)
    .reset_index()
    .rename(columns={c: f'shrug_{c}' for c in shrug_feature_cols})
)

# city to district mapping
city_to_district_state = {
    'agra': ('agra', 'uttar pradesh'),
    'ahmedabad': ('ahmedabad', 'gujarat'),
    'allahabad': ('allahabad', 'uttar pradesh'),
    'amritsar': ('amritsar', 'punjab'),
    'aurangabad': ('aurangabad', 'maharashtra'),
    'bengaluru': ('bangalore', 'karnataka'),
    'bhopal': ('bhopal', 'madhya pradesh'),
    'chennai': ('chennai', 'tamil nadu'),
    'delhi': ('new delhi', 'nct of delhi'),
    'dhanbad': ('dhanbad', 'jharkhand'),
    'faridabad': ('faridabad', 'haryana'),
    'hyderabad': ('hyderabad', 'andhra pradesh'),
    'indore': ('indore', 'madhya pradesh'),
    'jaipur': ('jaipur', 'rajasthan'),
    'kanpur': ('kanpur nagar', 'uttar pradesh'),
    'kolkata': ('kolkata', 'west bengal'),
    'lucknow': ('lucknow', 'uttar pradesh'),
    'ludhiana': ('ludhiana', 'punjab'),
    'meerut': ('meerut', 'uttar pradesh'),
    'mumbai': ('mumbai suburban', 'maharashtra'),
    'nagpur': ('nagpur', 'maharashtra'),
    'nashik': ('nashik', 'maharashtra'),
    'patna': ('patna', 'bihar'),
    'pune': ('pune', 'maharashtra'),
```

```
    'rajkot': ('rajkot', 'gujarat'),
    'ranchi': ('ranchi', 'jharkhand'),
    'srinagar': ('srinagar', 'jammu and kashmir'),
    'surat': ('surat', 'gujarat'),
    'vadodara': ('vadodara', 'gujarat'),
    'varanasi': ('varanasi', 'uttar pradesh'),
}

econ['district_norm'] = econ['city_norm'].map(lambda c: city_to_district_state.
 ↪get(c, (np.nan, np.nan))[0])
econ['state_norm'] = econ['city_norm'].map(lambda c: city_to_district_state.
 ↪get(c, (np.nan, np.nan))[1])
econ['district_key'] = econ['district_norm'] + '|' + econ['state_norm']

econ_agg = (
    econ[econ['district_key'].notna()]
    .groupby('district_key')
    .mean(numeric_only=True)
    .reset_index()
)

sector_numeric_cols = [
    c for c in sector_econ.select_dtypes(include=[np.number]).columns
    if c not in ['Dist Code', 'State Code']
]
sector_panel = sector_econ[['district_key_raw', 'Year'] + sector_numeric_cols].
 ↪copy()
sector_panel = sector_panel[sector_panel['district_key_raw'].notna()].rename(
    columns={'district_key_raw': 'district_key', 'Year': 'sector_year_label'}
)
sector_col_map = {
    c: 'sector_' + re.sub(r'[^0-9a-zA-Z]+', '_', c).strip('_').lower()
    for c in sector_numeric_cols
}
sector_panel = sector_panel.rename(columns=sector_col_map)

base_df = districts[['district_key', 'district_norm',
 'state_norm', 'state_census_code', 'district_census_code', 'geometry']].
 ↪drop_duplicates().copy()
base_proj = base_df.to_crs(epsg=3857)
centroids_proj = base_proj.geometry.centroid
centroids = gpd.GeoSeries(centroids_proj, crs='EPSG:3857').to_crs(epsg=4326)
base_df['centroid_lat'] = centroids.y.values
base_df['centroid_lon'] = centroids.x.values
base_df = base_df.drop(columns=['geometry'])

# Merge all sources into 1
```

```
data_combi_1 = base_df.merge(rail_aggreg,
on=['district_key', 'district_norm', 'state_norm'],
                             how='left')
data_combi_1 = data_combi_1.merge(demo_agg, on='district_key', how='left')
data_combi_1 = data_combi_1.merge(shrug_agg, on=['state_census_code',␣
 ↪'district_census_code'], how='left')
data_combi_1 = data_combi_1.merge(econ_agg, on='district_key', how='left')

# left join data_combi_1 with the district data
data_combi_2 = data_combi_1.merge(sector_panel, on='district_key', how='left')
df = data_combi_2.copy()

# Setting zero for districts with no assigned stops in the combined railway␣
 ↪dataset.
for col in ['station_count', 'weekly_trains', 'stop_events',␣
 ↪'train_type_count']:
    if col in df.columns:
        df[col] = df[col].fillna(0)

df['avg_latitude'] = df['avg_latitude'].fillna(df['centroid_lat'])
df['avg_longitude'] = df['avg_longitude'].fillna(df['centroid_lon'])
df['geo_point_count'] = df['geo_point_count'].fillna(df['geo_point_count'].
 ↪median())

df['shrug_available'] = df['shrug_tot_p'].notna().astype(int) if 'shrug_tot_p'␣
 ↪in df.columns else 0
df['econ_available'] = df['district_key'].isin(econ_agg['district_key']).
 ↪astype(int)
df['sector_econ_available'] = df['district_key'].
 ↪isin(sector_panel['district_key']).astype(int)

df['district'] = df['district_norm']
df['state'] = df['state_norm']

for col in df.select_dtypes(include=[np.number]).columns:
    df[col] = df[col].fillna(df[col].median())

df = df.replace([np.inf, -np.inf], np.nan)
for col in df.select_dtypes(include=[np.number]).columns:
    df[col] = df[col].fillna(0 if df[col].isna().all() else df[col].median())
print('Remaining NA values in df:', int(df.isna().sum().sum()))

data_combi_2 = df.copy()

print('Raw sectoral economy rows:', sector_econ.shape[0])
print('Sectoral economy panel rows after key cleanup:', sector_panel.shape[0])
```

```
print('data_combi_1 shape:', data_combi_1.shape)
print('data_combi_2 shape:', data_combi_2.shape)
print('Merged district-year shape used for MLR:', df.shape)
print('Districts included:', df['district_key'].nunique())
print('Districts with nonzero rail stations:', int((df['station_count'] > 0).
 ↪sum()))
print('Districts with shrug coverage:', int(df['shrug_available'].sum()))
print('shrug subdistrict observations available:',␣
 ↪shrug_subdist['subdistrict_key'].nunique())
print('Districts with city economy coverage:', int(df['econ_available'].sum()))
print('Districts with sectoral economy coverage:',␣
 ↪int(df['sector_econ_available'].sum()))
```

```
Remaining NA values in df: 414
Raw sectoral economy rows: 2149
Sectoral economy panel rows after key cleanup: 2149
data_combi_1 shape: (641, 47)
data_combi_2 shape: (2003, 69)
Merged district-year shape used for MLR: (2003, 69)
Districts included: 641
Districts with nonzero rail stations: 1816
Districts with shrug coverage: 22
shrug subdistrict observations available: 5804
Districts with city economy coverage: 156
Districts with sectoral economy coverage: 1589
```

1.6 5A. Checking Combined Schema

This cell prints the shape and column list of the merged table. It is a quick checkpoint before feature engineering, useful for confirming that the expected railway, demographic, SHRuG, and sectoral economy fields are present.

[7]:
```
# Combined dataset overview (run after df is created)
print(f'Combined dataset shape: {df.shape}')
print('\nCombined dataset columns:')
for i, col in enumerate(df.columns, 1):
    print(f'{i}. {col}')
```

```
Combined dataset shape: (2003, 69)

Combined dataset columns:
1. district_key
2. district_norm
3. state_norm
4. state_census_code
5. district_census_code
6. centroid_lat
7. centroid_lon
```

8. station_count
9. weekly_trains
10. stop_events
11. train_type_count
12. avg_train_distance
13. avg_duration_h
14. ac_train_share
15. sleeper_train_share
16. geo_point_count
17. avg_latitude
18. avg_longitude
19. shrug_secc_hh
20. shrug_tot_p
21. shrug_sc_share
22. shrug_st_share
23. shrug_ed_prim_share
24. shrug_ed_sec_share
25. shrug_ed_grad_share
26. shrug_land_own_share
27. shrug_inc_5k_plus_share
28. shrug_inc_10k_plus_share
29. shrug_salary_any_share
30. shrug_salary_gov_share
31. shrug_salary_priv_share
32. shrug_ag_inc_hh
33. shrug_veh_two_three_share
34. shrug_veh_four_share
35. shrug_wall_mat_solid_share
36. shrug_roof_mat_solid_share
37. shrug_num_room_mean
38. shrug_irr_share1
39. shrug_irr_share2
40. Year
41. R&D Expenditure (% of GDP)
42. Patents per 100,000 Inhabitants
43. Unemployment Rate (%)
44. Youth Unemployment Rate (%)
45. SME Employment (%)
46. Tourism Sector Employment (%)
47. ICT Sector Employment (%)
48. sector_year_label
49. sector_primary_sector_constant_prices_in_millions_rs
50. sector_primary_sector_current_prices_in_million_rs
51. sector_primary_sector_constant_shares_percent
52. sector_primary_sector_current_shares_percent
53. sector_secondary_sector_constant_prices_millions_in_rs
54. sector_secondary_sector_current_prices_millions_in_rs
55. sector_secondary_sector_constant_shares_percent

56. sector_secondary_sector_current_shares_percent
57. sector_tertiary_sector_constant_prices_millions_in_rs
58. sector_tertiary_sector_current_prices_millions_in_rs
59. sector_tertiary_sector_constant_shares_percent
60. sector_tertiary_sector_current_shares_percent
61. sector_total_constant_prices_in_millions_rs
62. sector_total_current_prices_in_millions_rs
63. sector_per_capita_current_prices_1000_in_rs
64. sector_sector_year
65. shrug_available
66. econ_available
67. sector_econ_available
68. district
69. state

1.7 6. Create Rail Access Features and the Station Target

Here, I would like to see what social, economic, industrial, railway service, and railway spatial access predictors contribute to the number of train stations in a district.

[8]:
```
# Derived rail access features
df['station_per_1000_pts'] = 1000 * df['station_count'] / df['geo_point_count'].
 ↪replace(0, np.nan)
df['train_per_1000_pts'] = 1000 * df['weekly_trains'] / df['geo_point_count'].
 ↪replace(0, np.nan)
df['stop_per_station'] = df['stop_events'] / df['station_count'].replace(0, np.
 ↪nan)

# Distance to nearest major hub.
df['major_hub_flag'] = df['econ_available'].fillna(0).astype(int)

district_points = gpd.GeoDataFrame(
    df[['district_key', 'centroid_lon', 'centroid_lat', 'major_hub_flag']].
 ↪copy(),
    geometry=gpd.points_from_xy(df['centroid_lon'], df['centroid_lat']),
    crs='EPSG:4326'
).to_crs(epsg=3857)

hub_points = district_points[district_points['major_hub_flag'] == 1].copy()

if len(hub_points) > 1:
    nearest_hub_distances = []
    nearest_hub_keys = []

    for _, row in district_points.iterrows():
        candidate_hubs = hub_points.copy()
```

```
        # If the district is itself a major hub, measure distance to the next␣
 ↪nearest hub.
        if row['major_hub_flag'] == 1:
            candidate_hubs = candidate_hubs[candidate_hubs['district_key'] !=␣
 ↪row['district_key']]

        distances_m = candidate_hubs.geometry.distance(row.geometry)
        nearest_idx = distances_m.idxmin()
        nearest_hub_distances.append(float(distances_m.loc[nearest_idx]) / 1000)
        nearest_hub_keys.append(candidate_hubs.loc[nearest_idx, 'district_key'])

    df['distance_to_nearest_major_hub_km'] = nearest_hub_distances
    df['nearest_major_hub_key'] = nearest_hub_keys
else:
    df['distance_to_nearest_major_hub_km'] = np.nan
    df['nearest_major_hub_key'] = np.nan

df['distance_to_nearest_major_hub_log'] = np.
 ↪log1p(df['distance_to_nearest_major_hub_km'])

df['rail_density'] = df['station_count'] / df['geo_point_count'].replace(0, np.
 ↪nan)
df['rail_density_log'] = np.log1p(df['station_per_1000_pts'])
df['service_intensity_log'] = np.log1p(df['train_per_1000_pts'])
df['access_potential_log'] = np.log1p(0.5 * df['station_per_1000_pts'] + 0.5 *␣
 ↪df['train_per_1000_pts'])

derived_cols = [
    'station_per_1000_pts', 'train_per_1000_pts', 'stop_per_station',
    'major_hub_flag', 'distance_to_nearest_major_hub_km',␣
 ↪'distance_to_nearest_major_hub_log',
    'rail_density', 'rail_density_log', 'service_intensity_log',␣
 ↪'access_potential_log'
]
for col in derived_cols:
    df[col] = df[col].replace([np.inf, -np.inf], np.nan).fillna(0)

df = df.replace([np.inf, -np.inf], np.nan)
for col in df.select_dtypes(include=[np.number]).columns:
    df[col] = df[col].fillna(0 if df[col].isna().all() else df[col].median())
data_combi_2 = df.copy()

print('Major hub districts:', int(df['major_hub_flag'].sum()))
print('Distance-to-hub summary, km:')
print(df['distance_to_nearest_major_hub_km'].describe())
print('data_combi_2 final shape used for MLR:', data_combi_2.shape)
```

```
Major hub districts: 156
Distance-to-hub summary, km:
count    2003.000000
mean      219.304496
std       175.170970
min         4.136744
25%       106.497020
50%       165.590630
75%       261.834252
max      1613.269234
Name: distance_to_nearest_major_hub_km, dtype: float64
data_combi_2 final shape used for MLR: (2003, 80)
```

1.8 7. Prepare the MLR Predictor Set

[9]:
```
target_col = 'station_count'
y = df[target_col]

requested_mlr_predictors = [
    'shrug_secc_hh',
    'shrug_tot_p',
    'shrug_sc_share',
    'shrug_st_share',
    'shrug_ed_prim_share',
    'shrug_ed_sec_share',
    'shrug_ed_grad_share',
    'shrug_land_own_share',
    'shrug_inc_5k_plus_share',
    'shrug_inc_10k_plus_share',
    'shrug_salary_any_share',
    'shrug_salary_gov_share',
    'shrug_salary_priv_share',
    'shrug_ag_inc_hh',
    'shrug_veh_two_three_share',
    'shrug_veh_four_share',
    'shrug_wall_mat_solid_share',
    'shrug_roof_mat_solid_share',
    'shrug_num_room_mean',
    'shrug_irr_share1',
    'shrug_irr_share2',
    'Year',
    'R&D Expenditure (% of GDP)',
    'Patents per 100,000 Inhabitants',
    'Unemployment Rate (%)',
    'Youth Unemployment Rate (%)',
    'SME Employment (%)',
    'Tourism Sector Employment (%)',
```

```
    'ICT Sector Employment (%)',
    'sector_year_label',
    'sector_primary_sector_constant_prices_in_millions_rs',
    'sector_primary_sector_current_prices_in_million_rs',
    'sector_primary_sector_constant_shares_percent',
    'sector_primary_sector_current_shares_percent',
    'sector_secondary_sector_constant_prices_millions_in_rs',
    'sector_secondary_sector_current_prices_millions_in_rs',
    'sector_secondary_sector_constant_shares_percent',
    'sector_secondary_sector_current_shares_percent',
    'sector_tertiary_sector_constant_prices_millions_in_rs',
    'sector_tertiary_sector_current_prices_millions_in_rs',
    'sector_tertiary_sector_constant_shares_percent',
    'sector_tertiary_sector_current_shares_percent',
    'sector_total_constant_prices_in_millions_rs',
    'sector_total_current_prices_in_millions_rs',
    'sector_per_capita_current_prices_1000_in_rs',
    'sector_sector_year',
    'shrug_available',
    'econ_available',
    'sector_econ_available',
    'major_hub_flag',
    'distance_to_nearest_major_hub_km',
    'distance_to_nearest_major_hub_log',
    'station_per_1000_pts',
    'train_per_1000_pts',
    'stop_per_station',
    'rail_density',
    'rail_density_log',
    'service_intensity_log',
    'access_potential_log',
]

mlr_predictors = [c for c in requested_mlr_predictors if c in df.columns]
missing_mlr_predictors = [c for c in requested_mlr_predictors if c not in df.
 ↪columns]

X_raw = df[mlr_predictors].copy()

X = pd.get_dummies(X_raw, columns=['sector_year_label'] if 'sector_year_label'␣
 ↪in X_raw.columns else [], drop_first=True)
for col in X.columns:
    X[col] = pd.to_numeric(X[col], errors='coerce')
X = X.replace([np.inf, -np.inf], np.nan)
for col in X.columns:
    X[col] = X[col].fillna(0 if X[col].isna().all() else X[col].median())
X = X.astype(float)
```

```
# Drop predictors that are constant
constant_mlr_predictors = [c for c in X.columns if X[c].nunique(dropna=False)␣
 ↪<= 1]
if constant_mlr_predictors:
    X = X.drop(columns=constant_mlr_predictors)

X = sm.add_constant(X, has_constant='add')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print('MLR target:', target_col)
print('Requested MLR predictors:', len(requested_mlr_predictors))
print(requested_mlr_predictors)
print('Predictors found before encoding:', len(mlr_predictors))
print('Predictor matrix columns after encoding/constant drop:', X.shape[1])
print('Missing requested predictors:', missing_mlr_predictors if␣
 ↪missing_mlr_predictors else 'None')
print('Constant predictors dropped:', constant_mlr_predictors if␣
 ↪constant_mlr_predictors else 'None')
```

```
MLR target: station_count
Requested MLR predictors: 59
['shrug_secc_hh', 'shrug_tot_p', 'shrug_sc_share', 'shrug_st_share',
'shrug_ed_prim_share', 'shrug_ed_sec_share', 'shrug_ed_grad_share',
'shrug_land_own_share', 'shrug_inc_5k_plus_share', 'shrug_inc_10k_plus_share',
'shrug_salary_any_share', 'shrug_salary_gov_share', 'shrug_salary_priv_share',
'shrug_ag_inc_hh', 'shrug_veh_two_three_share', 'shrug_veh_four_share',
'shrug_wall_mat_solid_share', 'shrug_roof_mat_solid_share',
'shrug_num_room_mean', 'shrug_irr_share1', 'shrug_irr_share2', 'Year', 'R&D
Expenditure (% of GDP)', 'Patents per 100,000 Inhabitants', 'Unemployment Rate
(%)', 'Youth Unemployment Rate (%)', 'SME Employment (%)', 'Tourism Sector
Employment (%)', 'ICT Sector Employment (%)', 'sector_year_label',
'sector_primary_sector_constant_prices_in_millions_rs',
'sector_primary_sector_current_prices_in_million_rs',
'sector_primary_sector_constant_shares_percent',
'sector_primary_sector_current_shares_percent',
'sector_secondary_sector_constant_prices_millions_in_rs',
'sector_secondary_sector_current_prices_millions_in_rs',
'sector_secondary_sector_constant_shares_percent',
'sector_secondary_sector_current_shares_percent',
'sector_tertiary_sector_constant_prices_millions_in_rs',
'sector_tertiary_sector_current_prices_millions_in_rs',
'sector_tertiary_sector_constant_shares_percent',
'sector_tertiary_sector_current_shares_percent',
```

```
'sector_total_constant_prices_in_millions_rs',
'sector_total_current_prices_in_millions_rs',
'sector_per_capita_current_prices_1000_in_rs', 'sector_sector_year',
'shrug_available', 'econ_available', 'sector_econ_available', 'major_hub_flag',
'distance_to_nearest_major_hub_km', 'distance_to_nearest_major_hub_log',
'station_per_1000_pts', 'train_per_1000_pts', 'stop_per_station',
'rail_density', 'rail_density_log', 'service_intensity_log',
'access_potential_log']
Predictors found before encoding: 59
Predictor matrix columns after encoding/constant drop: 64
Missing requested predictors: None
Constant predictors dropped: ['Year']
```

1.9 8. Fit the Model

[10]:
```
print('MLR dataset shape:', data_combi_2.shape)
print('MLR rows used:', X_train.shape[0] + X_test.shape[0])
print('MLR predictor matrix shape:', X.shape)
print('MLR train shape:', X_train.shape)
print('MLR test shape:', X_test.shape)
print('All columns in data_combi_2 used to build the MLR dataset:')
for i, col in enumerate(data_combi_2.columns, 1):
    print(f'{i}. {col}')
print('MLR predictors included:')
print([c for c in X.columns if c != 'const'])

model = sm.OLS(y_train, X_train).fit()
print(model.summary())
```

```
MLR dataset shape: (2003, 80)
MLR rows used: 2003
MLR predictor matrix shape: (2003, 64)
MLR train shape: (1602, 64)
MLR test shape: (401, 64)
All columns in data_combi_2 used to build the MLR dataset:
1. district_key
2. district_norm
3. state_norm
4. state_census_code
5. district_census_code
6. centroid_lat
7. centroid_lon
8. station_count
9. weekly_trains
10. stop_events
11. train_type_count
12. avg_train_distance
13. avg_duration_h
```

14. ac_train_share
15. sleeper_train_share
16. geo_point_count
17. avg_latitude
18. avg_longitude
19. shrug_secc_hh
20. shrug_tot_p
21. shrug_sc_share
22. shrug_st_share
23. shrug_ed_prim_share
24. shrug_ed_sec_share
25. shrug_ed_grad_share
26. shrug_land_own_share
27. shrug_inc_5k_plus_share
28. shrug_inc_10k_plus_share
29. shrug_salary_any_share
30. shrug_salary_gov_share
31. shrug_salary_priv_share
32. shrug_ag_inc_hh
33. shrug_veh_two_three_share
34. shrug_veh_four_share
35. shrug_wall_mat_solid_share
36. shrug_roof_mat_solid_share
37. shrug_num_room_mean
38. shrug_irr_share1
39. shrug_irr_share2
40. Year
41. R&D Expenditure (% of GDP)
42. Patents per 100,000 Inhabitants
43. Unemployment Rate (%)
44. Youth Unemployment Rate (%)
45. SME Employment (%)
46. Tourism Sector Employment (%)
47. ICT Sector Employment (%)
48. sector_year_label
49. sector_primary_sector_constant_prices_in_millions_rs
50. sector_primary_sector_current_prices_in_million_rs
51. sector_primary_sector_constant_shares_percent
52. sector_primary_sector_current_shares_percent
53. sector_secondary_sector_constant_prices_millions_in_rs
54. sector_secondary_sector_current_prices_millions_in_rs
55. sector_secondary_sector_constant_shares_percent
56. sector_secondary_sector_current_shares_percent
57. sector_tertiary_sector_constant_prices_millions_in_rs
58. sector_tertiary_sector_current_prices_millions_in_rs
59. sector_tertiary_sector_constant_shares_percent
60. sector_tertiary_sector_current_shares_percent
61. sector_total_constant_prices_in_millions_rs

62. sector_total_current_prices_in_millions_rs
63. sector_per_capita_current_prices_1000_in_rs
64. sector_sector_year
65. shrug_available
66. econ_available
67. sector_econ_available
68. district
69. state
70. station_per_1000_pts
71. train_per_1000_pts
72. stop_per_station
73. major_hub_flag
74. distance_to_nearest_major_hub_km
75. nearest_major_hub_key
76. distance_to_nearest_major_hub_log
77. rail_density
78. rail_density_log
79. service_intensity_log
80. access_potential_log

MLR predictors included:

['shrug_secc_hh', 'shrug_tot_p', 'shrug_sc_share', 'shrug_st_share',
'shrug_ed_prim_share', 'shrug_ed_sec_share', 'shrug_ed_grad_share',
'shrug_land_own_share', 'shrug_inc_5k_plus_share', 'shrug_inc_10k_plus_share',
'shrug_salary_any_share', 'shrug_salary_gov_share', 'shrug_salary_priv_share',
'shrug_ag_inc_hh', 'shrug_veh_two_three_share', 'shrug_veh_four_share',
'shrug_wall_mat_solid_share', 'shrug_roof_mat_solid_share',
'shrug_num_room_mean', 'shrug_irr_share1', 'shrug_irr_share2', 'R&D Expenditure
(% of GDP)', 'Patents per 100,000 Inhabitants', 'Unemployment Rate (%)', 'Youth
Unemployment Rate (%)', 'SME Employment (%)', 'Tourism Sector Employment (%)',
'ICT Sector Employment (%)',
'sector_primary_sector_constant_prices_in_millions_rs',
'sector_primary_sector_current_prices_in_million_rs',
'sector_primary_sector_constant_shares_percent',
'sector_primary_sector_current_shares_percent',
'sector_secondary_sector_constant_prices_millions_in_rs',
'sector_secondary_sector_current_prices_millions_in_rs',
'sector_secondary_sector_constant_shares_percent',
'sector_secondary_sector_current_shares_percent',
'sector_tertiary_sector_constant_prices_millions_in_rs',
'sector_tertiary_sector_current_prices_millions_in_rs',
'sector_tertiary_sector_constant_shares_percent',
'sector_tertiary_sector_current_shares_percent',
'sector_total_constant_prices_in_millions_rs',
'sector_total_current_prices_in_millions_rs',
'sector_per_capita_current_prices_1000_in_rs', 'sector_sector_year',
'shrug_available', 'econ_available', 'sector_econ_available', 'major_hub_flag',
'distance_to_nearest_major_hub_km', 'distance_to_nearest_major_hub_log',
'station_per_1000_pts', 'train_per_1000_pts', 'stop_per_station',

```
'rail_density', 'rail_density_log', 'service_intensity_log',
'access_potential_log', 'sector_year_label_2008 (2004)', 'sector_year_label_2009
(2004)', 'sector_year_label_2010 (2004)', 'sector_year_label_2011 (2004)',
'sector_year_label_2012 (2004)', 'sector_year_label_2013 (2004)']
                            OLS Regression Results
==============================================================================
Dep. Variable:          station_count   R-squared:                       0.738
Model:                            OLS   Adj. R-squared:                  0.728
Method:                 Least Squares   F-statistic:                     77.60
Date:                Sun, 26 Apr 2026   Prob (F-statistic):               0.00
Time:                        18:46:39   Log-Likelihood:                -5329.1
No. Observations:                1602   AIC:                         1.077e+04
Df Residuals:                    1545   BIC:                         1.108e+04
Df Model:                          56
Covariance Type:            nonrobust
==============================================================================
========================================
                                                    coef    std err
t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
----------------------------------------
const                                            1.2153      3.664
0.332      0.740      -5.972       8.402
shrug_secc_hh                                -9.971e-05      0.001
-0.090      0.928      -0.002       0.002
shrug_tot_p                                   1.867e-05      0.000
0.088      0.930      -0.000       0.000
shrug_sc_share                                 -34.0692    478.447
-0.071      0.943    -972.544     904.405
shrug_st_share                                  46.2252    343.781
0.134      0.893    -628.101     720.551
shrug_ed_prim_share                              0.7911    286.583
0.003      0.998    -561.342     562.925
shrug_ed_sec_share                              58.6651    631.385
0.093      0.926   -1179.797    1297.128
shrug_ed_grad_share                           -232.1004    772.948
-0.300      0.764   -1748.238    1284.038
shrug_land_own_share                            31.8550     86.324
0.369      0.712    -137.470     201.180
shrug_inc_5k_plus_share                        -57.1221    192.534
-0.297      0.767    -434.777     320.533
shrug_inc_10k_plus_share                       150.9202    402.856
0.375      0.708    -639.282     941.123
shrug_salary_any_share                          56.5666    170.003
0.333      0.739    -276.895     390.028
shrug_salary_gov_share                        -150.1761    505.204
-0.297      0.766   -1141.133     840.781
shrug_salary_priv_share                        120.4562    413.220
```

```
0.292      0.771    -690.076     930.988
shrug_ag_inc_hh                                          -0.0224      0.109
-0.205      0.838      -0.237       0.192
shrug_veh_two_three_share                               -16.7215    146.848
-0.114      0.909    -304.763     271.320
shrug_veh_four_share                                    -32.4557    196.792
-0.165      0.869    -418.463     353.552
shrug_wall_mat_solid_share                                7.0933     43.954
0.161      0.872     -79.122      93.309
shrug_roof_mat_solid_share                                7.0952     48.771
0.145      0.884     -88.569     102.760
shrug_num_room_mean                                      -2.0166     20.152
-0.100      0.920     -41.545      37.512
shrug_irr_share1                                         23.4543     61.813
0.379      0.704     -97.792     144.701
shrug_irr_share2                                         -9.3412     51.508
-0.181      0.856    -110.373      91.691
R&D Expenditure (% of GDP)                               -5.2799      1.585
-3.331      0.001      -8.389      -2.171
Patents per 100,000 Inhabitants                           1.4551      0.561
2.596      0.010       0.356       2.555
Unemployment Rate (%)                                     1.0221      0.525
1.947      0.052      -0.008       2.052
Youth Unemployment Rate (%)                              -6.8115      0.679
-10.025      0.000      -8.144      -5.479
SME Employment (%)                                       -0.5861      0.137
-4.275      0.000      -0.855      -0.317
Tourism Sector Employment (%)                            -1.3251      0.343
-3.869      0.000      -1.997      -0.653
ICT Sector Employment (%)                                 0.3787      0.141
2.687      0.007       0.102       0.655
sector_primary_sector_constant_prices_in_millions_rs     -0.0480      0.367
-0.131      0.896      -0.768       0.672
sector_primary_sector_current_prices_in_million_rs       -0.1699      0.338
-0.502      0.615      -0.833       0.493
sector_primary_sector_constant_shares_percent            90.7073     37.095
2.445      0.015      17.945     163.469
sector_primary_sector_current_shares_percent              0.7934     36.751
0.022      0.983     -71.294      72.881
sector_secondary_sector_constant_prices_millions_in_rs   -0.0486      0.367
-0.132      0.895      -0.769       0.672
sector_secondary_sector_current_prices_millions_in_rs    -0.1698      0.338
-0.502      0.616      -0.833       0.494
sector_secondary_sector_constant_shares_percent          90.4478     37.101
2.438      0.015      17.675     163.221
sector_secondary_sector_current_shares_percent            1.3087     36.756
0.036      0.972     -70.788      73.406
sector_tertiary_sector_constant_prices_millions_in_rs    -0.0487      0.367
```

```
-0.133      0.894      -0.769       0.671
sector_tertiary_sector_current_prices_millions_in_rs      -0.1697      0.338
-0.502      0.616      -0.833       0.494
sector_tertiary_sector_constant_shares_percent           91.0370     37.096
2.454      0.014      18.273     163.801
sector_tertiary_sector_current_shares_percent             0.7437     36.748
0.020      0.984     -71.338      72.825
sector_total_constant_prices_in_millions_rs               0.0487      0.367
0.133      0.894      -0.671       0.769
sector_total_current_prices_in_millions_rs                0.1697      0.338
0.502      0.616      -0.494       0.833
sector_per_capita_current_prices_1000_in_rs              -0.0880      0.008
-10.620      0.000      -0.104      -0.072
sector_sector_year                                       -5.0311      2.936
-1.714      0.087     -10.789       0.727
shrug_available                                          -2.5084     13.063
-0.192      0.848     -28.132      23.115
econ_available                                            0.8531      0.486
1.755      0.080      -0.101       1.807
sector_econ_available                                   984.6499   2502.900
0.393      0.694   -3924.789    5894.089
major_hub_flag                                            0.8531      0.486
1.755      0.080      -0.101       1.807
distance_to_nearest_major_hub_km                          0.0048      0.002
2.037      0.042       0.000       0.009
distance_to_nearest_major_hub_log                        -2.4887      0.602
-4.136      0.000      -3.669      -1.309
station_per_1000_pts                                      0.1198      0.015
7.951      0.000       0.090       0.149
train_per_1000_pts                                       -0.0134      0.002
-7.964      0.000      -0.017      -0.010
stop_per_station                                          0.0909      0.009
10.633      0.000       0.074       0.108
rail_density                                              0.0001   1.51e-05
7.951      0.000    9.02e-05       0.000
rail_density_log                                         10.4453      0.804
12.991      0.000       8.868      12.022
service_intensity_log                                     0.5126      1.988
0.258      0.797      -3.386       4.412
access_potential_log                                     -6.1624      2.654
-2.322      0.020     -11.369      -0.956
sector_year_label_2008 (2004)                             5.5612      3.025
1.838      0.066      -0.372      11.495
sector_year_label_2009 (2004)                            10.6333      5.903
1.801      0.072      -0.946      22.213
sector_year_label_2010 (2004)                            16.5777      8.831
1.877      0.061      -0.745      33.900
sector_year_label_2011 (2004)                            22.3095     11.755
```

```
1.898      0.058      -0.748      45.367
sector_year_label_2012 (2004)                          27.7765     14.706
1.889      0.059      -1.069      56.622
sector_year_label_2013 (2004)                          33.5955     17.606
1.908      0.057      -0.938      68.130
==============================================================================
Omnibus:                      317.245   Durbin-Watson:                   2.025
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1063.081
Skew:                           0.963   Prob(JB):                    1.43e-231
Kurtosis:                       6.495   Cond. No.                     9.90e+17
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The smallest eigenvalue is 3.48e-22. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

1.10 8A. Forward and Backward Stepwise Regression

[ ]:
```python
trainData = X_train.copy()
trainData[target_col] = y_train

candidate_predictors = [col for col in X_train.columns if col != 'const']


def fit_ols_for_predictors(predictors):
    """Fit an OLS model on trainData using an intercept plus the selected␣
 ↪predictors."""
    if predictors:
        X_step = sm.add_constant(trainData[predictors], has_constant='add')
    else:
        X_step = pd.DataFrame({'const': np.ones(len(trainData))},␣
 ↪index=trainData.index)
    return sm.OLS(trainData[target_col], X_step).fit()


def forward_stepwise_aic(candidates):
    """Start with no predictors and add the predictor that most improves AIC."""
    selected = []
    remaining = list(candidates)
    current_model = fit_ols_for_predictors(selected)

    while remaining:
        trial_results = []
        for predictor in remaining:
            trial_predictors = selected + [predictor]
```

```
            trial_model = fit_ols_for_predictors(trial_predictors)
            trial_results.append((trial_model.aic, predictor, trial_model))

        best_aic, best_predictor, best_model = min(trial_results, key=lambda␣
 ↪item: item[0])
        if best_aic < current_model.aic:
            selected.append(best_predictor)
            remaining.remove(best_predictor)
            current_model = best_model
        else:
            break

    return selected, current_model


def backward_stepwise_aic(candidates):
    """Start with all predictors and remove the predictor that most improves␣
 ↪AIC."""
    selected = list(candidates)
    current_model = fit_ols_for_predictors(selected)

    while len(selected) > 1:
        trial_results = []
        for predictor in selected:
            trial_predictors = [p for p in selected if p != predictor]
            trial_model = fit_ols_for_predictors(trial_predictors)
            trial_results.append((trial_model.aic, predictor, trial_model,␣
 ↪trial_predictors))

        best_aic, removed_predictor, best_model, best_predictors =␣
 ↪min(trial_results, key=lambda item: item[0])
        if best_aic < current_model.aic:
            selected = best_predictors
            current_model = best_model
        else:
            break

    return selected, current_model


# Forward stepwise
forward_selected_variables, stepwise_forward =␣
 ↪forward_stepwise_aic(candidate_predictors)

print('Forward Stepwise Selected Model Summary')
print(stepwise_forward.summary())
print()
```

```
print('Forward Stepwise Selected Variables:')
print(forward_selected_variables)
print('Forward Stepwise AIC:', stepwise_forward.aic)
print('Forward Stepwise BIC:', stepwise_forward.bic)

forward_stepwise_results = pd.DataFrame({
    'model': ['stepwise_forward'],
    'selected_variables': [', '.join(forward_selected_variables)],
    'AIC': [stepwise_forward.aic],
    'BIC': [stepwise_forward.bic],
})
forward_stepwise_results
```

```
Forward Stepwise Selected Model Summary
                            OLS Regression Results
==============================================================================
Dep. Variable:          station_count   R-squared:                       0.736
Model:                            OLS   Adj. R-squared:                  0.731
Method:                 Least Squares   F-statistic:                     162.3
Date:                Sun, 26 Apr 2026   Prob (F-statistic):               0.00
Time:                        18:47:00   Log-Likelihood:                -5335.1
No. Observations:                1602   AIC:                         1.073e+04
Df Residuals:                    1574   BIC:                         1.088e+04
Df Model:                          27
Covariance Type:            nonrobust
==============================================================================
                                                    coef    std err
t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const                                          -934.0587    305.308
-3.059      0.002   -1532.912    -335.206
rail_density_log                                 10.2010      0.558
18.289      0.000       9.107      11.295
sector_primary_sector_constant_prices_in_millions_rs       0.0007   7.28e-05
9.557      0.000       0.001       0.001
sector_primary_sector_current_shares_percent       0.1147      0.115
0.999      0.318      -0.110       0.340
Youth Unemployment Rate (%)                      -6.7709      0.667
-10.147      0.000      -8.080      -5.462
access_potential_log                             -5.4560      0.457
-11.928      0.000      -6.353      -4.559
stop_per_station                                  0.0901      0.008
10.824      0.000       0.074       0.106
sector_total_constant_prices_in_millions_rs      1.768e-05   1.53e-05
1.157      0.247   -1.23e-05    4.77e-05
sector_per_capita_current_prices_1000_in_rs      -0.0882      0.008
```

```
-10.866      0.000      -0.104      -0.072
sector_econ_available                                      2.3060      0.567
4.068      0.000       1.194       3.418
distance_to_nearest_major_hub_log                         -2.3992      0.588
-4.082      0.000      -3.552      -1.246
ICT Sector Employment (%)                                  0.3822      0.139
2.756      0.006       0.110       0.654
distance_to_nearest_major_hub_km                           0.0047      0.002
2.015      0.044       0.000       0.009
SME Employment (%)                                        -0.5910      0.133
-4.452      0.000      -0.851      -0.331
Tourism Sector Employment (%)                             -1.2754      0.335
-3.803      0.000      -1.933      -0.618
sector_tertiary_sector_current_prices_millions_in_rs   -4.558e-08   8.41e-06
-0.005      0.996   -1.65e-05    1.64e-05
rail_density                                               0.0001   1.47e-05
8.300      0.000    9.32e-05       0.000
train_per_1000_pts                                        -0.0137      0.002
-8.322      0.000      -0.017      -0.010
Patents per 100,000 Inhabitants                            1.5849      0.550
2.884      0.004       0.507       2.663
R&D Expenditure (% of GDP)                                -5.3669      1.553
-3.456      0.001      -8.413      -2.321
sector_sector_year                                         0.4923      0.152
3.244      0.001       0.195       0.790
sector_primary_sector_current_prices_in_million_rs        -0.0002   3.44e-05
-4.537      0.000      -0.000   -8.86e-05
sector_tertiary_sector_constant_shares_percent             0.4055      0.115
3.537      0.000       0.181       0.630
sector_secondary_sector_current_shares_percent             0.3785      0.123
3.078      0.002       0.137       0.620
sector_secondary_sector_constant_prices_millions_in_rs  4.799e-05   2.43e-05
1.978      0.048    3.89e-07    9.56e-05
Unemployment Rate (%)                                      0.9723      0.517
1.880      0.060      -0.042       1.987
econ_available                                             1.8016      0.936
1.925      0.054      -0.034       3.637
shrug_available                                           -2.5566      1.703
-1.501      0.134      -5.898       0.785
station_per_1000_pts                                       0.1220      0.015
8.300      0.000       0.093       0.151
==============================================================================
Omnibus:                      319.985   Durbin-Watson:                   2.028
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1065.881
Skew:                           0.973   Prob(JB):                    3.52e-232
Kurtosis:                       6.490   Cond. No.                     1.23e+16
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The smallest eigenvalue is 6.07e-19. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.

Forward Stepwise Selected Variables:
['rail_density_log', 'sector_primary_sector_constant_prices_in_millions_rs',
'sector_primary_sector_current_shares_percent', 'Youth Unemployment Rate (%)',
'access_potential_log', 'stop_per_station',
'sector_total_constant_prices_in_millions_rs',
'sector_per_capita_current_prices_1000_in_rs', 'sector_econ_available',
'distance_to_nearest_major_hub_log', 'ICT Sector Employment (%)',
'distance_to_nearest_major_hub_km', 'SME Employment (%)', 'Tourism Sector
Employment (%)', 'sector_tertiary_sector_current_prices_millions_in_rs',
'rail_density', 'train_per_1000_pts', 'Patents per 100,000 Inhabitants', 'R&D
Expenditure (% of GDP)', 'sector_sector_year',
'sector_primary_sector_current_prices_in_million_rs',
'sector_tertiary_sector_constant_shares_percent',
'sector_secondary_sector_current_shares_percent',
'sector_secondary_sector_constant_prices_millions_in_rs', 'Unemployment Rate
(%)', 'econ_available', 'shrug_available', 'station_per_1000_pts']
Forward Stepwise AIC: 10726.145138360527
Forward Stepwise BIC: 10876.75736593412
```

[ ]:

	model	selected_variables	\
0	stepwise_forward	rail_density_log, sector_primary_sector_consta…	

	AIC	BIC
0	10726.145138	10876.757366

1.10.1 Backward Stepwise Regression

[13]:
```
# Backward stepwise
backward_selected_variables, stepwise_backward =␣
 ↪backward_stepwise_aic(candidate_predictors)

print('Backward Stepwise Selected Model Summary')
print(stepwise_backward.summary())
print()
print('Backward Stepwise Selected Variables:')
print(backward_selected_variables)
print('Backward Stepwise AIC:', stepwise_backward.aic)
print('Backward Stepwise BIC:', stepwise_backward.bic)

backward_stepwise_results = pd.DataFrame({
    'model': ['stepwise_backward'],
```

```
    'selected_variables': [', '.join(backward_selected_variables)],
    'AIC': [stepwise_backward.aic],
    'BIC': [stepwise_backward.bic],
})
backward_stepwise_results
```

```
Backward Stepwise Selected Model Summary
                            OLS Regression Results
==============================================================================
Dep. Variable:          station_count   R-squared:                       0.737
Model:                            OLS   Adj. R-squared:                  0.732
Method:                 Least Squares   F-statistic:                     137.6
Date:                Sun, 26 Apr 2026   Prob (F-statistic):               0.00
Time:                        18:52:45   Log-Likelihood:                -5330.4
No. Observations:                1602   AIC:                         1.073e+04
Df Residuals:                    1569   BIC:                         1.090e+04
Df Model:                          32
Covariance Type:            nonrobust
==============================================================================
                                                                 coef    std err
t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------
const                                                      -2.071e+05   8.28e+04
-2.501      0.012    -3.7e+05   -4.47e+04
R&D Expenditure (% of GDP)                                   -5.1402      1.543
-3.331      0.001      -8.167      -2.113
Patents per 100,000 Inhabitants                               1.5182      0.547
2.774      0.006       0.445       2.592
Unemployment Rate (%)                                         0.9774      0.515
1.900      0.058      -0.032       1.987
Youth Unemployment Rate (%)                                  -6.7310      0.668
-10.079      0.000      -8.041      -5.421
SME Employment (%)                                           -0.5872      0.132
-4.465      0.000      -0.845      -0.329
Tourism Sector Employment (%)                                -1.3025      0.336
-3.872      0.000      -1.962      -0.643
ICT Sector Employment (%)                                     0.3685      0.138
2.664      0.008       0.097       0.640
sector_primary_sector_current_prices_in_million_rs           -0.0001   3.27e-05
-4.131      0.000      -0.000    -7.1e-05
sector_primary_sector_constant_shares_percent                88.9927     35.266
2.523      0.012      19.820     158.166
sector_secondary_sector_constant_prices_millions_in_rs       -0.0006   6.89e-05
-8.855      0.000      -0.001      -0.000
sector_secondary_sector_constant_shares_percent              88.7638     35.270
2.517      0.012      19.582     157.946
```

```
sector_secondary_sector_current_shares_percent             0.4804      0.148
3.252      0.001       0.191       0.770
sector_tertiary_sector_constant_prices_millions_in_rs     -0.0007   6.78e-05
-9.749      0.000      -0.001      -0.001
sector_tertiary_sector_constant_shares_percent            89.2756     35.266
2.531      0.011      20.102     158.449
sector_total_constant_prices_in_millions_rs                0.0007   6.74e-05
10.061      0.000       0.001       0.001
sector_per_capita_current_prices_1000_in_rs               -0.0894      0.008
-11.030      0.000      -0.105      -0.074
sector_sector_year                                        98.7948     39.499
2.501      0.012      21.318     176.272
shrug_available                                           -2.5367      1.701
-1.491      0.136      -5.873       0.800
major_hub_flag                                             1.6579      0.923
1.796      0.073      -0.153       3.469
distance_to_nearest_major_hub_km                           0.0047      0.002
2.037      0.042       0.000       0.009
distance_to_nearest_major_hub_log                         -2.4732      0.588
-4.207      0.000      -3.626      -1.320
train_per_1000_pts                                        -0.0134      0.002
-8.170      0.000      -0.017      -0.010
stop_per_station                                           0.0905      0.008
10.904      0.000       0.074       0.107
rail_density                                             119.7086     14.697
8.145      0.000      90.881     148.536
rail_density_log                                          10.3055      0.559
18.427      0.000       9.209      11.403
access_potential_log                                      -5.4998      0.458
-12.001      0.000      -6.399      -4.601
sector_year_label_2008 (2004)                            -98.2897     39.524
-2.487      0.013    -175.814     -20.765
sector_year_label_2009 (2004)                           -197.0699     79.012
-2.494      0.013    -352.049     -42.090
sector_year_label_2010 (2004)                           -294.9622    118.491
-2.489      0.013    -527.379     -62.546
sector_year_label_2011 (2004)                           -393.1116    157.991
-2.488      0.013    -703.008     -83.215
sector_year_label_2012 (2004)                           -491.5349    197.485
-2.489      0.013    -878.897    -104.173
sector_year_label_2013 (2004)                           -589.5790    237.022
-2.487      0.013   -1054.492    -124.666
==============================================================================
Omnibus:                      317.290   Durbin-Watson:                   2.022
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1056.787
Skew:                           0.965   Prob(JB):                    3.32e-230
Kurtosis:                       6.479   Cond. No.                     1.01e+11
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 1.01e+11. This might indicate that there are
strong multicollinearity or other numerical problems.

Backward Stepwise Selected Variables:
['R&D Expenditure (% of GDP)', 'Patents per 100,000 Inhabitants', 'Unemployment
Rate (%)', 'Youth Unemployment Rate (%)', 'SME Employment (%)', 'Tourism Sector
Employment (%)', 'ICT Sector Employment (%)',
'sector_primary_sector_current_prices_in_million_rs',
'sector_primary_sector_constant_shares_percent',
'sector_secondary_sector_constant_prices_millions_in_rs',
'sector_secondary_sector_constant_shares_percent',
'sector_secondary_sector_current_shares_percent',
'sector_tertiary_sector_constant_prices_millions_in_rs',
'sector_tertiary_sector_constant_shares_percent',
'sector_total_constant_prices_in_millions_rs',
'sector_per_capita_current_prices_1000_in_rs', 'sector_sector_year',
'shrug_available', 'major_hub_flag', 'distance_to_nearest_major_hub_km',
'distance_to_nearest_major_hub_log', 'train_per_1000_pts', 'stop_per_station',
'rail_density', 'rail_density_log', 'access_potential_log',
'sector_year_label_2008 (2004)', 'sector_year_label_2009 (2004)',
'sector_year_label_2010 (2004)', 'sector_year_label_2011 (2004)',
'sector_year_label_2012 (2004)', 'sector_year_label_2013 (2004)']
Backward Stepwise AIC: 10726.878991821559
Backward Stepwise BIC: 10904.386260033292
```

[13]:

	model	selected_variables	\
0	stepwise_backward	R&D Expenditure (% of GDP), Patents per 100,00…	

	AIC	BIC
0	10726.878992	10904.38626

1.10.2 Refit and Interpret the Combined Stepwise Model

[28]:
```
# Combine predictors
combined_stepwise_predictors = list(dict.fromkeys(
    forward_selected_variables + backward_selected_variables
))

X_train_stepwise_combined = sm.add_constant(
    trainData[combined_stepwise_predictors],
    has_constant='add'
)
```

```
stepwise_selected_model = sm.OLS(trainData[target_col],␣
 ↪X_train_stepwise_combined).fit()

analysis_model = stepwise_selected_model
analysis_model_name = 'Combined Stepwise OLS'
analysis_predictors = combined_stepwise_predictors
X_train_analysis = X_train_stepwise_combined
X_test_analysis = sm.add_constant(X_test[analysis_predictors],␣
 ↪has_constant='add')
X_analysis = sm.add_constant(X[analysis_predictors], has_constant='add')

print('Combined stepwise predictors selected by either forward or backward AIC␣
 ↪search:')
print(combined_stepwise_predictors)
print('Combined Stepwise Selected Model Summary')
print(stepwise_selected_model.summary())

# Interpretation table
stepwise_interpretation = pd.DataFrame({
    'predictor': stepwise_selected_model.params.index,
    'coefficient': stepwise_selected_model.params.values,
    'p_value': stepwise_selected_model.pvalues.values,
})
stepwise_interpretation =␣
 ↪stepwise_interpretation[stepwise_interpretation['predictor'] != 'const'].
 ↪copy()
stepwise_interpretation['direction'] = np.where(
    stepwise_interpretation['coefficient'] > 0,
    'positive association with station_count',
    'negative association with station_count'
)
stepwise_interpretation['significant_at_0_05'] =␣
 ↪stepwise_interpretation['p_value'] < 0.05
stepwise_interpretation = stepwise_interpretation.sort_values('p_value')

print('Interpretation table sorted by p-value:')
display(stepwise_interpretation)

print('Model fit:')
print('AIC:', stepwise_selected_model.aic)
print('BIC:', stepwise_selected_model.bic)
print('Adjusted R-squared:', stepwise_selected_model.rsquared_adj)

significant_predictors =␣
 ↪stepwise_interpretation[stepwise_interpretation['significant_at_0_05']].
 ↪copy()
```

```
positive_predictors =␣
 ↪significant_predictors[significant_predictors['coefficient'] > 0]
negative_predictors =␣
 ↪significant_predictors[significant_predictors['coefficient'] < 0]

if len(significant_predictors) == 0:
    print('No selected predictors are statistically significant at alpha = 0.05.
 ↪ In that case, the stepwise model is useful for prediction/model reduction,␣
 ↪but individual coefficient interpretation should be cautious.')
else:
    print(f'{len(significant_predictors)} selected predictors are statistically␣
 ↪significant at alpha = 0.05.')

    if len(positive_predictors) > 0:
        print()
        print('Significant positive contributors to station_count:')
        for _, row in positive_predictors.iterrows():
            print(
                f"- {row['predictor']}: coefficient = {row['coefficient']:.4f},␣
 ↪"
                f"p-value = {row['p_value']:.4g}. Higher values are associated␣
 ↪with more stations."
            )

    if len(negative_predictors) > 0:
        print()
        print('Significant negative contributors to station_count:')
        for _, row in negative_predictors.iterrows():
            print(
                f"- {row['predictor']}: coefficient = {row['coefficient']:.4f},␣
 ↪"
                f"p-value = {row['p_value']:.4g}. Higher values are associated␣
 ↪with fewer stations."
            )
```

```
Combined stepwise predictors selected by either forward or backward AIC search:
['rail_density_log', 'sector_primary_sector_constant_prices_in_millions_rs',
'sector_primary_sector_current_shares_percent', 'Youth Unemployment Rate (%)',
'access_potential_log', 'stop_per_station',
'sector_total_constant_prices_in_millions_rs',
'sector_per_capita_current_prices_1000_in_rs', 'sector_econ_available',
'distance_to_nearest_major_hub_log', 'ICT Sector Employment (%)',
'distance_to_nearest_major_hub_km', 'SME Employment (%)', 'Tourism Sector
Employment (%)', 'sector_tertiary_sector_current_prices_millions_in_rs',
'rail_density', 'train_per_1000_pts', 'Patents per 100,000 Inhabitants', 'R&D
Expenditure (% of GDP)', 'sector_sector_year',
'sector_primary_sector_current_prices_in_million_rs',
```

```
'sector_tertiary_sector_constant_shares_percent',
'sector_secondary_sector_current_shares_percent',
'sector_secondary_sector_constant_prices_millions_in_rs', 'Unemployment Rate
(%)', 'econ_available', 'shrug_available', 'station_per_1000_pts',
'sector_primary_sector_constant_shares_percent',
'sector_secondary_sector_constant_shares_percent',
'sector_tertiary_sector_constant_prices_millions_in_rs', 'major_hub_flag',
'sector_year_label_2008 (2004)', 'sector_year_label_2009 (2004)',
'sector_year_label_2010 (2004)', 'sector_year_label_2011 (2004)',
'sector_year_label_2012 (2004)', 'sector_year_label_2013 (2004)']
Combined Stepwise Selected Model Summary
                            OLS Regression Results
==============================================================================
Dep. Variable:          station_count   R-squared:                       0.737
Model:                            OLS   Adj. R-squared:                  0.731
Method:                 Least Squares   F-statistic:                     125.6
Date:                Sun, 26 Apr 2026   Prob (F-statistic):               0.00
Time:                        19:47:40   Log-Likelihood:                -5330.3
No. Observations:                1602   AIC:                         1.073e+04
Df Residuals:                    1566   BIC:                         1.093e+04
Df Model:                          35
Covariance Type:            nonrobust
==============================================================================
==========================================
                                                                  coef    std err
t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------
const                                                          -0.5139      1.189
-0.432      0.666      -2.846       1.818
rail_density_log                                               10.2834      0.562
18.297      0.000       9.181      11.386
sector_primary_sector_constant_prices_in_millions_rs           -0.0519      0.365
-0.142      0.887      -0.767       0.663
sector_primary_sector_current_shares_percent                    0.0561      0.127
0.442      0.658      -0.193       0.305
Youth Unemployment Rate (%)                                    -6.7415      0.669
-10.078      0.000      -8.054      -5.429
access_potential_log                                           -5.4845      0.460
-11.927      0.000      -6.386      -4.583
stop_per_station                                                0.0902      0.008
10.802      0.000       0.074       0.107
sector_total_constant_prices_in_millions_rs                     0.0526      0.365
0.144      0.885      -0.663       0.768
sector_per_capita_current_prices_1000_in_rs                    -0.0894      0.008
-10.995      0.000      -0.105      -0.073
sector_econ_available                                        -194.9987    813.697
-0.240      0.811   -1791.049    1401.051
```

```
distance_to_nearest_major_hub_log                              -2.4547      0.590
-4.163      0.000      -3.611      -1.298
ICT Sector Employment (%)                                       0.3705      0.139
2.665      0.008       0.098       0.643
distance_to_nearest_major_hub_km                                0.0047      0.002
2.001      0.046    9.17e-05       0.009
SME Employment (%)                                             -0.5881      0.133
-4.430      0.000      -0.849      -0.328
Tourism Sector Employment (%)                                  -1.3015      0.337
-3.859      0.000      -1.963      -0.640
sector_tertiary_sector_current_prices_millions_in_rs   -5.714e-07   8.42e-06
-0.068      0.946   -1.71e-05    1.59e-05
rail_density                                                    0.0001   1.48e-05
8.119      0.000    9.09e-05       0.000
train_per_1000_pts                                             -0.0135      0.002
-8.171      0.000      -0.017      -0.010
Patents per 100,000 Inhabitants                                 1.5409      0.551
2.796      0.005       0.460       2.622
R&D Expenditure (% of GDP)                                     -5.1345      1.558
-3.295      0.001      -8.191      -2.078
sector_sector_year                                             -4.3795      1.919
-2.282      0.023      -8.143      -0.616
sector_primary_sector_current_prices_in_million_rs             -0.0001   3.76e-05
-3.805      0.000      -0.000   -6.94e-05
sector_tertiary_sector_constant_shares_percent                 90.8236     36.665
2.477      0.013      18.905     162.742
sector_secondary_sector_current_shares_percent                  0.5077      0.163
3.111      0.002       0.188       0.828
sector_secondary_sector_constant_prices_millions_in_rs         -0.0525      0.365
-0.144      0.885      -0.768       0.663
Unemployment Rate (%)                                           0.9783      0.519
1.887      0.059      -0.039       1.995
econ_available                                                  0.8488      0.468
1.813      0.070      -0.069       1.767
shrug_available                                                -2.5463      1.703
-1.495      0.135      -5.887       0.794
station_per_1000_pts                                            0.1199      0.015
8.119      0.000       0.091       0.149
sector_primary_sector_constant_shares_percent                  90.4853     36.665
2.468      0.014      18.568     162.403
sector_secondary_sector_constant_shares_percent                90.2887     36.668
2.462      0.014      18.366     162.212
sector_tertiary_sector_constant_prices_millions_in_rs          -0.0526      0.365
-0.144      0.885      -0.768       0.663
major_hub_flag                                                  0.8488      0.468
1.813      0.070      -0.069       1.767
sector_year_label_2008 (2004)                                   4.8597      2.031
2.393      0.017       0.876       8.843
```

```
sector_year_label_2009 (2004)                            9.2503      3.899
2.373      0.018       1.603      16.898
sector_year_label_2010 (2004)                           14.5211      5.816
2.497      0.013       3.114      25.928
sector_year_label_2011 (2004)                           19.5304      7.716
2.531      0.011       4.395      34.665
sector_year_label_2012 (2004)                           24.3210      9.647
2.521      0.012       5.398      43.244
sector_year_label_2013 (2004)                           29.4711     11.527
2.557      0.011       6.861      52.081
==============================================================================
Omnibus:                      317.409   Durbin-Watson:                   2.022
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1059.495
Skew:                           0.965   Prob(JB):                    8.58e-231
Kurtosis:                       6.485   Cond. No.                     6.24e+17
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The smallest eigenvalue is 2.81e-22. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
Interpretation table sorted by p-value:

                                          predictor   coefficient  \
1                                  rail_density_log  1.028337e+01
5                               access_potential_log -5.484530e+00
8        sector_per_capita_current_prices_1000_in_rs -8.935678e-02
6                                  stop_per_station  9.022050e-02
4                        Youth Unemployment Rate (%) -6.741481e+00
17                              train_per_1000_pts -1.347332e-02
28                             station_per_1000_pts  1.199024e-01
16                                    rail_density  1.199024e-04
13                               SME Employment (%) -5.881351e-01
10                distance_to_nearest_major_hub_log -2.454734e+00
14                    Tourism Sector Employment (%) -1.301489e+00
21  sector_primary_sector_current_prices_in_millio… -1.431661e-04
19                       R&D Expenditure (% of GDP) -5.134505e+00
23    sector_secondary_sector_current_shares_percent  5.077391e-01
18                  Patents per 100,000 Inhabitants  1.540877e+00
11                       ICT Sector Employment (%)  3.704923e-01
38                   sector_year_label_2013 (2004)  2.947106e+01
36                   sector_year_label_2011 (2004)  1.953037e+01
37                   sector_year_label_2012 (2004)  2.432096e+01
35                   sector_year_label_2010 (2004)  1.452108e+01
22    sector_tertiary_sector_constant_shares_percent  9.082361e+01
29     sector_primary_sector_constant_shares_percent  9.048529e+01
30   sector_secondary_sector_constant_shares_percent  9.028872e+01
```

33	sector_year_label_2008 (2004)	4.859729e+00
34	sector_year_label_2009 (2004)	9.250262e+00
20	sector_sector_year	-4.379531e+00
12	distance_to_nearest_major_hub_km	4.674329e-03
25	Unemployment Rate (%)	9.782991e-01
26	econ_available	8.488241e-01
32	major_hub_flag	8.488241e-01
27	shrug_available	-2.546255e+00
3	sector_primary_sector_current_shares_percent	5.614425e-02
9	sector_econ_available	-1.949987e+02
7	sector_total_constant_prices_in_millions_rs	5.261330e-02
31	sector_tertiary_sector_constant_prices_million…	-5.259508e-02
24	sector_secondary_sector_constant_prices_millio…	-5.254626e-02
2	sector_primary_sector_constant_prices_in_milli…	-5.192236e-02
15	sector_tertiary_sector_current_prices_millions…	-5.713988e-07

	p_value	direction	significant_at_0_05
1	6.284056e-68	positive association with station_count	True
5	1.880635e-31	negative association with station_count	True
8	3.861525e-27	negative association with station_count	True
6	2.760804e-26	positive association with station_count	True
4	3.443096e-23	negative association with station_count	True
17	6.214059e-16	negative association with station_count	True
28	9.431245e-16	positive association with station_count	True
16	9.431268e-16	positive association with station_count	True
13	1.006674e-05	negative association with station_count	True
10	3.313149e-05	negative association with station_count	True
14	1.183140e-04	negative association with station_count	True
21	1.470205e-04	negative association with station_count	True
19	1.004872e-03	negative association with station_count	True
23	1.895204e-03	positive association with station_count	True
18	5.236259e-03	positive association with station_count	True
11	7.785049e-03	positive association with station_count	True
38	1.066129e-02	positive association with station_count	True
36	1.146687e-02	positive association with station_count	True
37	1.180137e-02	positive association with station_count	True
35	1.262875e-02	positive association with station_count	True
22	1.335077e-02	positive association with station_count	True
29	1.369677e-02	positive association with station_count	True
30	1.391032e-02	positive association with station_count	True
33	1.683306e-02	positive association with station_count	True
34	1.778572e-02	positive association with station_count	True
20	2.259499e-02	negative association with station_count	True
12	4.559328e-02	positive association with station_count	True
25	5.937671e-02	positive association with station_count	False
26	7.000436e-02	positive association with station_count	False
32	7.000436e-02	positive association with station_count	False
27	1.350632e-01	negative association with station_count	False

```
3   6.582912e-01  positive association with station_count               False
9   8.106365e-01  negative association with station_count               False
7   8.853039e-01  positive association with station_count               False
31  8.853432e-01  negative association with station_count               False
24  8.854491e-01  negative association with station_count               False
2   8.867995e-01  negative association with station_count               False
15  9.459086e-01  negative association with station_count               False

Model fit:
AIC: 10732.627388372824
BIC: 10926.271680967444
Adjusted R-squared: 0.7314638748463558
27 selected predictors are statistically significant at alpha = 0.05.

Significant positive contributors to station_count:
- rail_density_log: coefficient = 10.2834, p-value = 6.284e-68. Higher values
are associated with more stations.
- stop_per_station: coefficient = 0.0902, p-value = 2.761e-26. Higher values are
associated with more stations.
- station_per_1000_pts: coefficient = 0.1199, p-value = 9.431e-16. Higher values
are associated with more stations.
- rail_density: coefficient = 0.0001, p-value = 9.431e-16. Higher values are
associated with more stations.
- sector_secondary_sector_current_shares_percent: coefficient = 0.5077, p-value
= 0.001895. Higher values are associated with more stations.
- Patents per 100,000 Inhabitants: coefficient = 1.5409, p-value = 0.005236.
Higher values are associated with more stations.
- ICT Sector Employment (%): coefficient = 0.3705, p-value = 0.007785. Higher
values are associated with more stations.
- sector_year_label_2013 (2004): coefficient = 29.4711, p-value = 0.01066.
Higher values are associated with more stations.
- sector_year_label_2011 (2004): coefficient = 19.5304, p-value = 0.01147.
Higher values are associated with more stations.
- sector_year_label_2012 (2004): coefficient = 24.3210, p-value = 0.0118. Higher
values are associated with more stations.
- sector_year_label_2010 (2004): coefficient = 14.5211, p-value = 0.01263.
Higher values are associated with more stations.
- sector_tertiary_sector_constant_shares_percent: coefficient = 90.8236, p-value
= 0.01335. Higher values are associated with more stations.
- sector_primary_sector_constant_shares_percent: coefficient = 90.4853, p-value
= 0.0137. Higher values are associated with more stations.
- sector_secondary_sector_constant_shares_percent: coefficient = 90.2887,
p-value = 0.01391. Higher values are associated with more stations.
- sector_year_label_2008 (2004): coefficient = 4.8597, p-value = 0.01683. Higher
values are associated with more stations.
- sector_year_label_2009 (2004): coefficient = 9.2503, p-value = 0.01779. Higher
values are associated with more stations.
- distance_to_nearest_major_hub_km: coefficient = 0.0047, p-value = 0.04559.
```

```
Higher values are associated with more stations.

Significant negative contributors to station_count:
- access_potential_log: coefficient = -5.4845, p-value = 1.881e-31. Higher
values are associated with fewer stations.
- sector_per_capita_current_prices_1000_in_rs: coefficient = -0.0894, p-value =
3.862e-27. Higher values are associated with fewer stations.
- Youth Unemployment Rate (%): coefficient = -6.7415, p-value = 3.443e-23.
Higher values are associated with fewer stations.
- train_per_1000_pts: coefficient = -0.0135, p-value = 6.214e-16. Higher values
are associated with fewer stations.
- SME Employment (%): coefficient = -0.5881, p-value = 1.007e-05. Higher values
are associated with fewer stations.
- distance_to_nearest_major_hub_log: coefficient = -2.4547, p-value = 3.313e-05.
Higher values are associated with fewer stations.
- Tourism Sector Employment (%): coefficient = -1.3015, p-value = 0.0001183.
Higher values are associated with fewer stations.
- sector_primary_sector_current_prices_in_million_rs: coefficient = -0.0001,
p-value = 0.000147. Higher values are associated with fewer stations.
- R&D Expenditure (% of GDP): coefficient = -5.1345, p-value = 0.001005. Higher
values are associated with fewer stations.
- sector_sector_year: coefficient = -4.3795, p-value = 0.02259. Higher values
are associated with fewer stations.
```

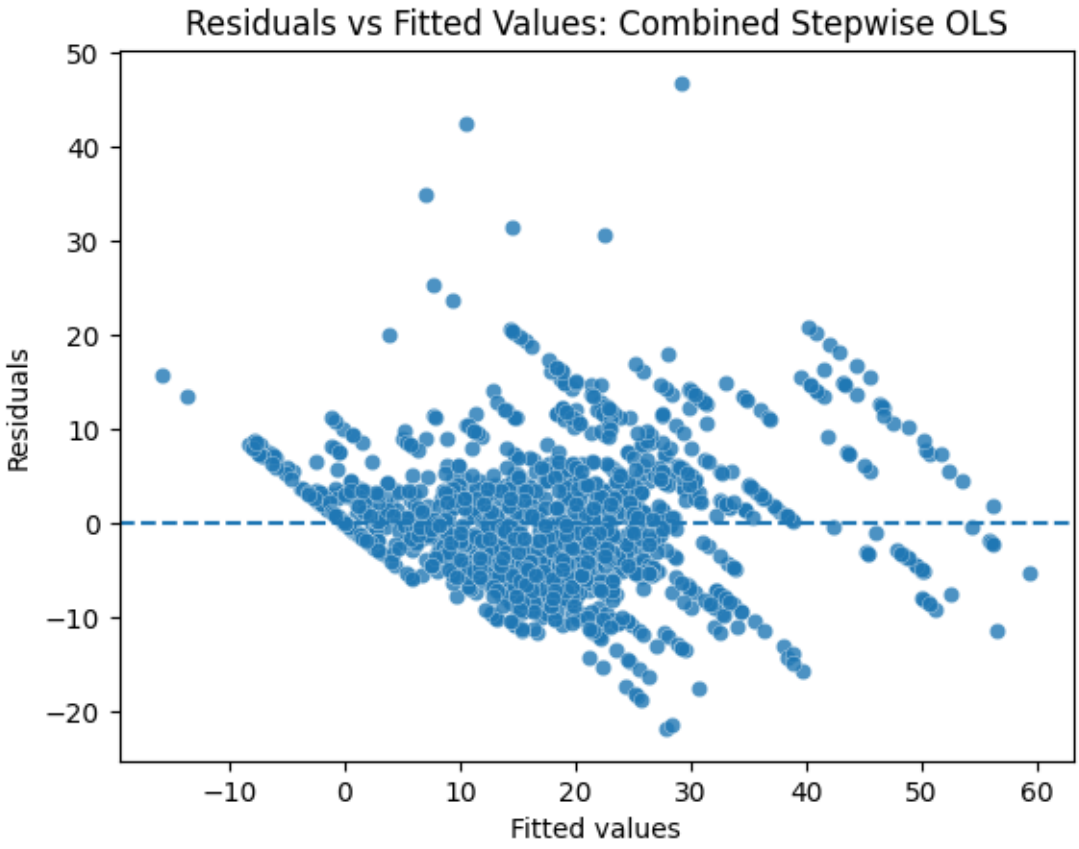
1.11 9. Check Residual Behavior

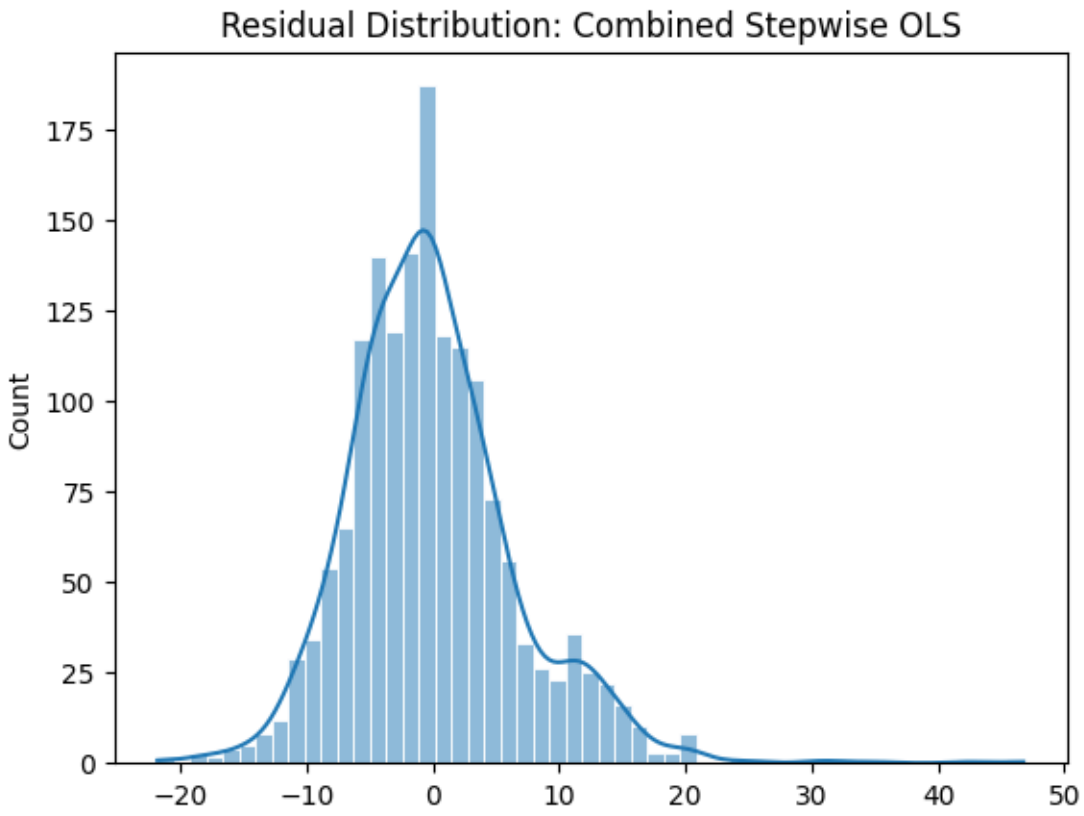
Residual plots and formal tests now use the combined stepwise model because it is the selected interpretation model. This matters because later rail-desert flags use the gap between actual and predicted access from this final selected model.

[33]:
```python
# Residual diagnostics for the selected combined stepwise model
residuals = analysis_model.resid
fitted_values = analysis_model.fittedvalues

print('Residual diagnostics for:', analysis_model_name)

# Linearity
sns.scatterplot(x=fitted_values, y=residuals, edgecolor='white', linewidth=0.
 ↪25, alpha=0.8)
plt.axhline(0, linestyle='--')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.title(f'Residuals vs Fitted Values: {analysis_model_name}')
plt.show()

# Normality
sns.histplot(residuals, kde=True, edgecolor='white')
plt.title(f'Residual Distribution: {analysis_model_name}')
```

```
plt.show()

# Jarque-Bera
print('JB:', jarque_bera(residuals))

# Heteroskedasticity
print('BP Test:', het_breuschpagan(residuals, X_train_analysis))
```

```
Residual diagnostics for: Combined Stepwise OLS
```






```
JB: SignificanceResult(statistic=np.float64(1059.4946745730952),
pvalue=np.float64(8.58330635282942e-231))
BP Test: (np.float64(154.54931340873375), np.float64(5.387768695065582e-16),
np.float64(4.777349525916038), np.float64(5.40001553159806e-18))
```

1.12 10. Examine Variable Distributions

[37]:
```python
# Distribution summary for numeric variables used in modeling
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
dist_summary = pd.DataFrame({
    'feature': numeric_cols,
    'mean': [df[c].mean() for c in numeric_cols],
    'std': [df[c].std() for c in numeric_cols],
    'skewness': [skew(df[c].dropna()) for c in numeric_cols],
    'kurtosis': [kurtosis(df[c].dropna()) for c in numeric_cols],
})
print(dist_summary.sort_values('skewness', key=np.abs, ascending=False))

# Histograms for target and key rail accessibility components
plot_cols = [
    target_col,
```

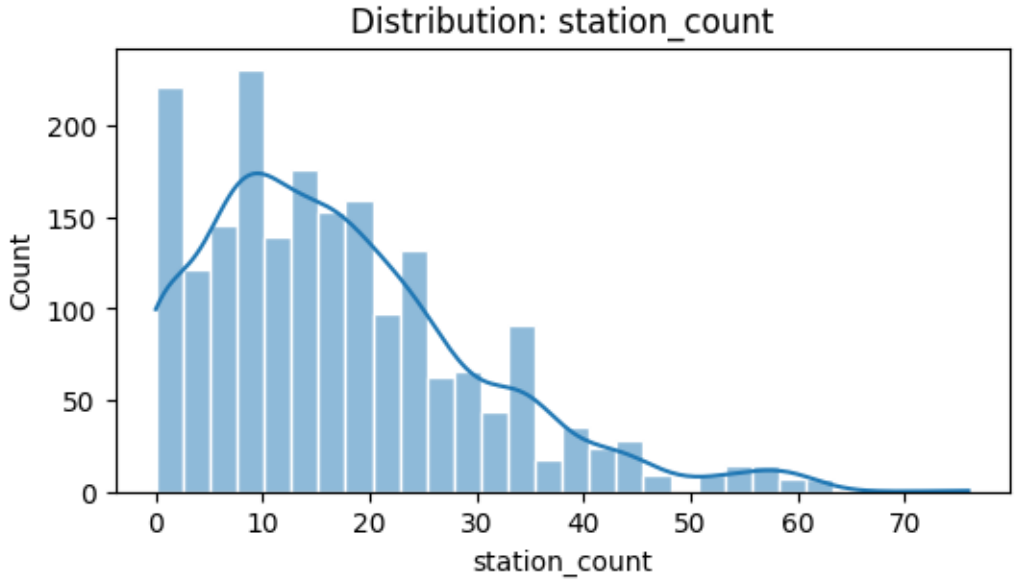
```
    'station_per_1000_pts',
    'train_per_1000_pts',
    'stop_per_station',
]
plot_cols = [c for c in plot_cols if c in df.columns]
for col in plot_cols:
    plt.figure(figsize=(6, 3))
    sns.histplot(df[col].dropna(), kde=True, edgecolor='white')
    plt.title(f'Distribution: {col}')
    plt.show()

# QQ plot + normality checks on residuals
sm.qqplot(residuals, line='45')
plt.gca().get_lines()[0].set_markerfacecolor(cold_color)
plt.gca().get_lines()[0].set_markeredgecolor(cold_color)
plt.gca().get_lines()[1].set_color(hot_color)
plt.title('QQ Plot of Residuals')
plt.show()

print('Residual skewness:', skew(residuals))
print('Residual kurtosis:', kurtosis(residuals))
if len(residuals) >= 3 and len(residuals) <= 5000:
    print('Shapiro-Wilk:', shapiro(residuals))
```

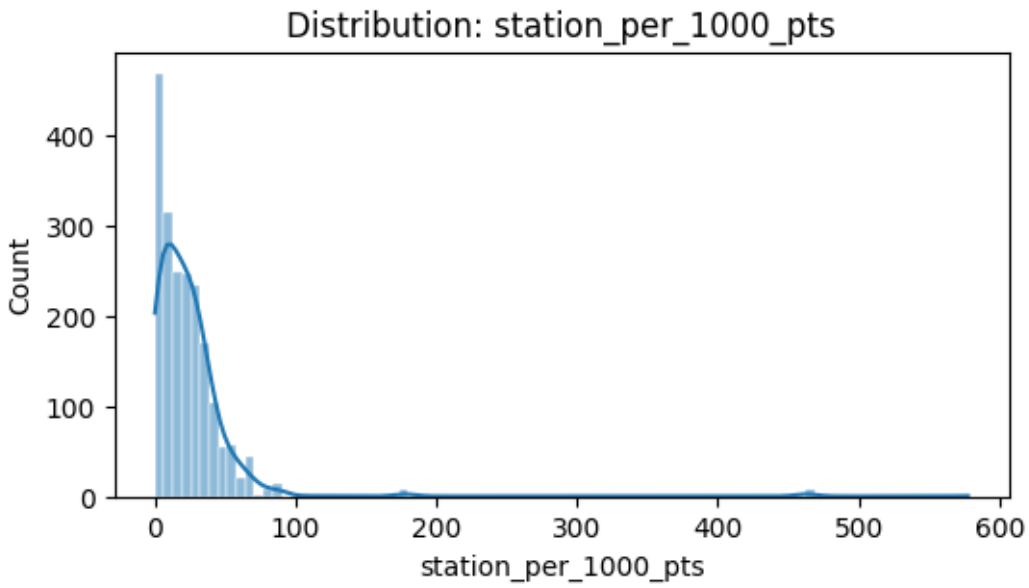
```
/tmp/ipykernel_30670/575603712.py:7: RuntimeWarning: Precision loss occurred in
moment calculation due to catastrophic cancellation. This occurs when the data
are nearly identical. Results may be unreliable.
  'skewness': [skew(df[c].dropna()) for c in numeric_cols],
/tmp/ipykernel_30670/575603712.py:8: RuntimeWarning: Precision loss occurred in
moment calculation due to catastrophic cancellation. This occurs when the data
are nearly identical. Results may be unreliable.
  'kurtosis': [kurtosis(df[c].dropna()) for c in numeric_cols],
```

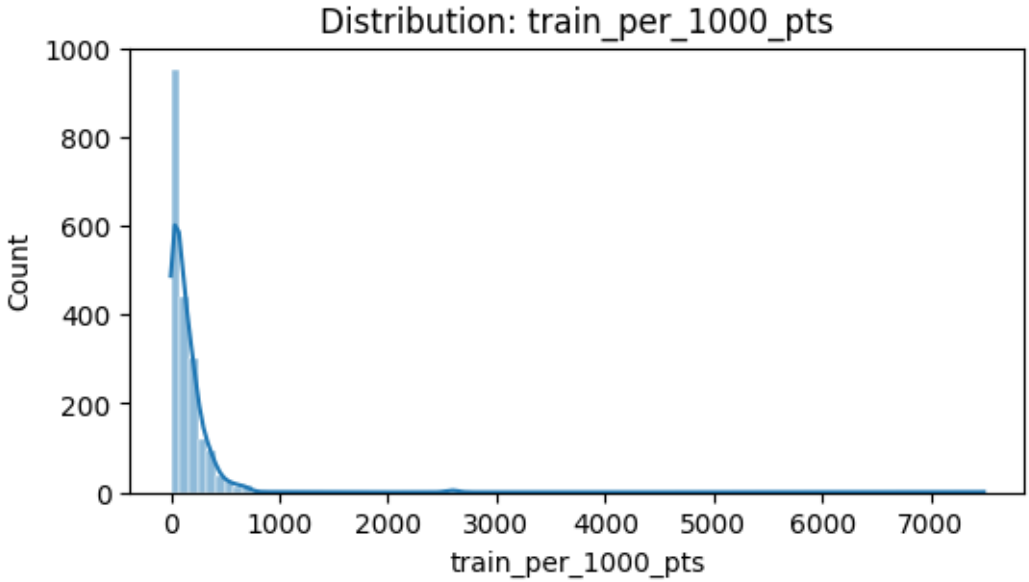
	feature	mean	std	\
18	shrug_st_share	0.035494	0.009499	
27	shrug_salary_priv_share	0.010988	0.001910	
29	shrug_veh_two_three_share	0.037894	0.013010	
17	shrug_sc_share	0.001087	0.004703	
25	shrug_salary_any_share	0.200283	0.008672	
..	…	…	…	
7	train_type_count	5.332002	2.986334	
68	distance_to_nearest_major_hub_log	5.156894	0.679422	
47	sector_primary_sector_current_shares_percent	25.947589	10.615460	
59	sector_sector_year	2010.000000	1.781804	
36	Year	2024.000000	0.000000	

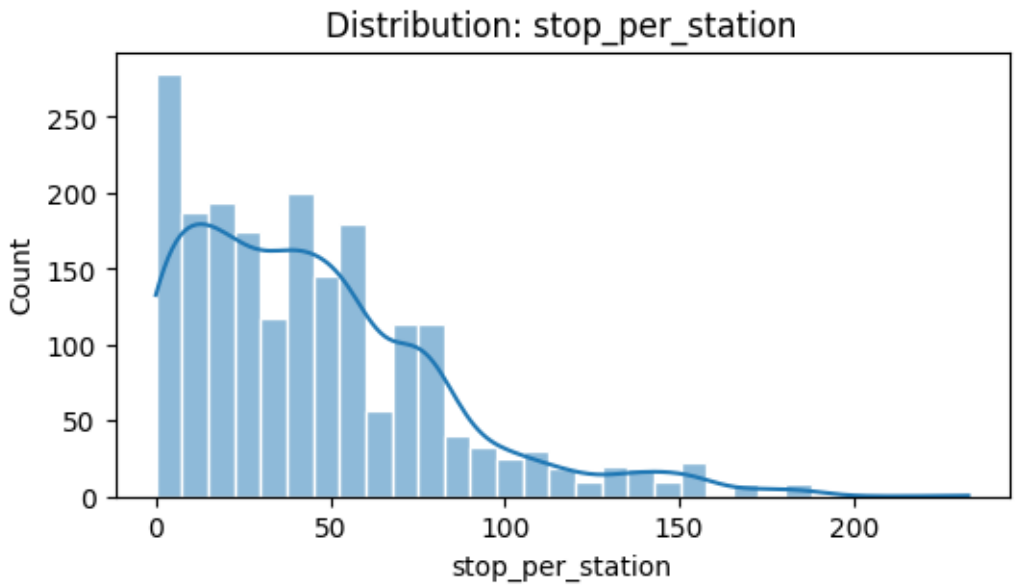
	skewness	kurtosis
18	26.294759	756.303120

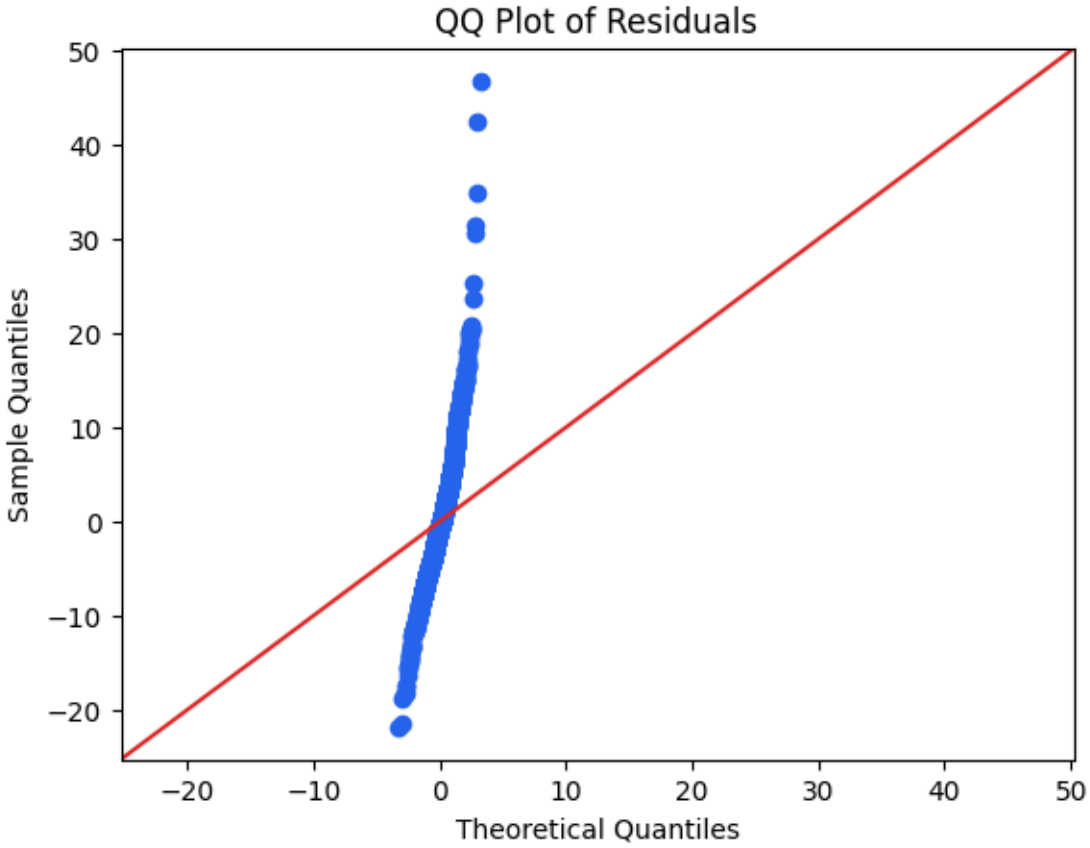
```
27   24.188292   669.901425
29   22.862660   565.168335
17   20.970243   473.849410
25   19.639260   499.080852
..         …           …
7    -0.122037    -0.844936
68    0.106637     0.524196
47    0.092870     0.276822
59    0.000000    -0.794053
36         NaN          NaN

[77 rows x 5 columns]
```




Distribution: station_per_1000_pts
Count
0
100
200
300
400
0
100
200
300
400
500
600
station_per_1000_pts


Distribution: train_per_1000_pts
Count
0
200
400
600
800
1000
0
1000
2000
3000
4000
5000
6000
7000
train_per_1000_pts


Distribution: stop_per_station
Count
stop_per_station
0
50
100
150
200
250


QQ Plot of Residuals
Sample Quantiles
Theoretical Quantiles
−20
−10
0
10
20
30
40
50

```
Residual skewness: 0.9649826035879104
Residual kurtosis: 3.485371385296591
Shapiro-Wilk: ShapiroResult(statistic=np.float64(0.9547579286491046),
pvalue=np.float64(7.301100300261219e-22))
```

1.13 11. Test Overall and Incremental Model Fit

[ ]:
```
if np.isfinite(analysis_model.fvalue) and np.isfinite(analysis_model.f_pvalue):
    print(f'{analysis_model_name} F-statistic:', analysis_model.fvalue)
    print(f'{analysis_model_name} p-value:', analysis_model.f_pvalue)
else:
    print(f'{analysis_model_name} F-test is undefined (likely due to small␣
 ↪sample and/or rank deficiency).')

baseline_candidates = [
    'shrug_tot_p', 'shrug_sc_share', 'shrug_st_share',
    'Year', 'Unemployment Rate (%)',␣
 ↪'sector_per_capita_current_prices_1000_in_rs',
    'distance_to_nearest_major_hub_km', 'distance_to_nearest_major_hub_log'
]
baseline_vars = [c for c in baseline_candidates if c in analysis_predictors]

if len(baseline_vars) > 0:
    X_train_reduced = sm.add_constant(X_train[baseline_vars],␣
 ↪has_constant='add')
    reduced_model = sm.OLS(y_train, X_train_reduced).fit()

    print('Partial F-test (Reduced baseline vs Combined Stepwise):')
    try:
        compare = anova_lm(reduced_model, analysis_model)
        print(compare)
    except Exception as e:
        print(f'Could not compute ANOVA partial F-test: {e}')

    extra_vars = [c for c in analysis_predictors if c not in baseline_vars]
    if extra_vars:
        hypothesis = ' = 0, '.join(extra_vars) + ' = 0'
        print('Joint F-test for selected non-baseline vars:', hypothesis)
        try:
            print(analysis_model.f_test(hypothesis))
        except Exception as e:
            print(f'Could not compute joint F-test (often rank/small-sample␣
 ↪issue): {e}')
else:
    print('No baseline variables from the compact list were selected into the␣
 ↪combined stepwise model.')
```

```
Combined Stepwise OLS F-statistic: 125.59868300070879
```

```
Combined Stepwise OLS p-value: 0.0
Partial F-test (Reduced baseline vs Combined Stepwise):
   df_resid            ssr  df_diff        ss_diff           F  Pr(>F)
0    1597.0  258447.832800      0.0            NaN         NaN     NaN
1    1566.0   72817.017343     31.0  185630.815457  128.779653     0.0
Joint F-test for selected non-baseline vars: rail_density_log = 0,
sector_primary_sector_constant_prices_in_millions_rs = 0,
sector_primary_sector_current_shares_percent = 0, Youth Unemployment Rate (%) =
0, access_potential_log = 0, stop_per_station = 0,
sector_total_constant_prices_in_millions_rs = 0, sector_econ_available = 0, ICT
Sector Employment (%) = 0, SME Employment (%) = 0, Tourism Sector Employment (%)
= 0, sector_tertiary_sector_current_prices_millions_in_rs = 0, rail_density = 0,
train_per_1000_pts = 0, Patents per 100,000 Inhabitants = 0, R&D Expenditure (%
of GDP) = 0, sector_sector_year = 0,
sector_primary_sector_current_prices_in_million_rs = 0,
sector_tertiary_sector_constant_shares_percent = 0,
sector_secondary_sector_current_shares_percent = 0,
sector_secondary_sector_constant_prices_millions_in_rs = 0, econ_available = 0,
shrug_available = 0, station_per_1000_pts = 0,
sector_primary_sector_constant_shares_percent = 0,
sector_secondary_sector_constant_shares_percent = 0,
sector_tertiary_sector_constant_prices_millions_in_rs = 0, major_hub_flag = 0,
sector_year_label_2008 (2004) = 0, sector_year_label_2009 (2004) = 0,
sector_year_label_2010 (2004) = 0, sector_year_label_2011 (2004) = 0,
sector_year_label_2012 (2004) = 0, sector_year_label_2013 (2004) = 0
<F test: F=np.float64(83.78495453801943), p=1.4864745474640856e-280,
df_denom=1.57e+03, df_num=27>
```

```
/home/aps834009/HW/.venv/lib/python3.11/site-
packages/statsmodels/base/model.py:1894: ValueWarning: covariance of constraints
does not have full rank. The number of constraints is 34, but rank is 27
  warnings.warn('covariance of constraints does not have full '
```

```
The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the␣
↪failure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more info.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further details.
```

1.14 12. Check Multicollinearity with VIF

VIF is now calculated on the selected combined stepwise model design matrix. This gives a cleaner multicollinearity check for the predictors being interpreted, instead of the much larger original

predictor set.

[39]:
```
vif_data = pd.DataFrame()
vif_data['feature'] = X_train_analysis.columns
vif_data['VIF'] = [
    variance_inflation_factor(X_train_analysis.values, i)
    for i in range(X_train_analysis.shape[1])
]
print(vif_data.sort_values('VIF', ascending=False))
```

```
/home/aps834009/HW/.venv/lib/python3.11/site-
packages/statsmodels/regression/linear_model.py:1782: RuntimeWarning: divide by
zero encountered in scalar divide
  return 1 - self.ssr/self.centered_tss
/home/aps834009/HW/.venv/lib/python3.11/site-
packages/statsmodels/stats/outliers_influence.py:197: RuntimeWarning: divide by
zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

	feature	VIF
28	station_per_1000_pts	inf
36	sector_year_label_2011 (2004)	inf
37	sector_year_label_2012 (2004)	inf
35	sector_year_label_2010 (2004)	inf
20	sector_sector_year	inf
16	rail_density	inf
9	sector_econ_available	inf
32	major_hub_flag	inf
33	sector_year_label_2008 (2004)	inf
34	sector_year_label_2009 (2004)	inf
26	econ_available	inf
38	sector_year_label_2013 (2004)	inf
7	sector_total_constant_prices_in_millions_rs	8.717094e+10
31	sector_tertiary_sector_constant_prices_million…	3.888514e+10
24	sector_secondary_sector_constant_prices_millio…	1.042631e+10
2	sector_primary_sector_constant_prices_in_milli…	4.845757e+08
29	sector_primary_sector_constant_shares_percent	4.409541e+06
30	sector_secondary_sector_constant_shares_percent	3.930090e+06
22	sector_tertiary_sector_constant_shares_percent	3.742538e+06
23	sector_secondary_sector_current_shares_percent	7.907352e+01
3	sector_primary_sector_current_shares_percent	6.323352e+01
15	sector_tertiary_sector_current_prices_millions…	4.908945e+01
21	sector_primary_sector_current_prices_in_millio…	2.454539e+01
5	access_potential_log	1.684528e+01
1	rail_density_log	1.523520e+01
17	train_per_1000_pts	7.008657e+00
12	distance_to_nearest_major_hub_km	5.648240e+00
10	distance_to_nearest_major_hub_log	5.442697e+00
8	sector_per_capita_current_prices_1000_in_rs	3.234530e+00

```
6                                   stop_per_station  3.172025e+00
13                                SME Employment (%)  2.166601e+00
18                 Patents per 100,000 Inhabitants  1.972357e+00
11                       ICT Sector Employment (%)  1.765952e+00
14                  Tourism Sector Employment (%)  1.708385e+00
19                     R&D Expenditure (% of GDP)  1.582856e+00
4                    Youth Unemployment Rate (%)  1.569917e+00
25                         Unemployment Rate (%)  1.318503e+00
27                               shrug_available  1.110013e+00
0                                          const  0.000000e+00
```

1.15 12. Interpret the VIF Results

This cell translates the VIF table into practical modeling guidance. If several predictors are highly collinear, coefficient signs and magnitudes should be interpreted carefully.

[43]:
```
vif_sorted = vif_data.sort_values('VIF', ascending=False).copy()

high_vif = vif_sorted[(vif_sorted['feature'] != 'const') & (vif_sorted['VIF'] >␣
 ↪10)]
moderate_vif = vif_sorted[(vif_sorted['feature'] != 'const') &␣
 ↪(vif_sorted['VIF'] > 5) & (vif_sorted['VIF'] <= 10)]
print(moderate_vif)
```

```
                            feature       VIF
17                train_per_1000_pts  7.008657
12   distance_to_nearest_major_hub_km  5.648240
10  distance_to_nearest_major_hub_log  5.442697
```

1.16 13. Validate the Selected Stepwise Model

Holdout and cross-validation metrics now evaluate the combined stepwise model. This is the main check before using its predictions to create access gaps and rail-desert rankings.

[51]:
```
pred = analysis_model.predict(X_test_analysis)

rmse = float(np.sqrt(mean_squared_error(y_test, pred)))
mae = float(mean_absolute_error(y_test, pred))
r2 = float(r2_score(y_test, pred))

denom = np.where(np.abs(y_test.values) < 1e-8, np.nan, np.abs(y_test.values))
mape = float(np.nanmean(np.abs((y_test.values - pred.values) / denom)) * 100)

print('Target:', target_col)
print(f'Holdout ({analysis_model_name})')
print('R2:', r2)
print('RMSE:', rmse)
print('MAE:', mae)
print('MAPE (%):', mape)
```

```
baseline_pred = np.repeat(y_train.mean(), len(y_test))
baseline_rmse = float(np.sqrt(mean_squared_error(y_test, baseline_pred)))
print('Baseline RMSE (predict train mean):', baseline_rmse)
print('RMSE Improvement vs Baseline (%):', 100 * (baseline_rmse - rmse) /␣
 ↪baseline_rmse)

X_cv = X_analysis.copy()
y_cv = y.copy()

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_rmse = []
cv_r2 = []

for train_idx, test_idx in kf.split(X_cv):
    X_tr = X_cv.iloc[train_idx]
    X_te = X_cv.iloc[test_idx]
    y_tr = y_cv.iloc[train_idx]
    y_te = y_cv.iloc[test_idx]

    m = sm.OLS(y_tr, X_tr).fit()
    p = m.predict(X_te)

    cv_rmse.append(np.sqrt(mean_squared_error(y_te, p)))
    cv_r2.append(r2_score(y_te, p))

print()
print(f'5-Fold CV ({analysis_model_name})')
print('CV RMSE mean +/- std:', float(np.mean(cv_rmse)), '+/-', float(np.
 ↪std(cv_rmse)))
print('CV R2 mean +/- std:', float(np.mean(cv_r2)), '+/-', float(np.std(cv_r2)))
```

```
Target: station_count
Holdout (Combined Stepwise OLS)
R2: 0.7269892449522264
RMSE: 6.141604421223029
MAE: 4.7221264585601395
MAPE (%): 44.90659841161967
Baseline RMSE (predict train mean): 11.778286868085251
RMSE Improvement vs Baseline (%): 47.85655596600828

5-Fold CV (Combined Stepwise OLS)
CV RMSE mean +/- std: 6.892134330327053 +/- 0.45668304855171854
CV R2 mean +/- std: 0.7125649290587798 +/- 0.02412021897311035
```

1.17 13A. Compare Candidate Model

[52]:
```
model_rows = []

pred_original = model.predict(X_test)
model_rows.append({
    'model_name': 'Original Full OLS',
    'r2': float(r2_score(y_test, pred_original)),
    'rmse': float(np.sqrt(mean_squared_error(y_test, pred_original))),
    'holdout_mae': float(mean_absolute_error(y_test, pred_original)),
})

pred_stepwise = analysis_model.predict(X_test_analysis)
model_rows.append({
    'model_name': analysis_model_name,
    'r2': float(r2_score(y_test, pred_stepwise)),
    'rmse': float(np.sqrt(mean_squared_error(y_test, pred_stepwise))),
    'mae': float(mean_absolute_error(y_test, pred_stepwise)),
})

if 'model_reduced_vif' in globals() and 'X_test_reduced_vif' in globals():
    pred_reduced = model_reduced_vif.predict(X_test_reduced_vif)
    model_rows.append({
        'model_name': 'Reduced-VIF Selected OLS',
        'r2': float(r2_score(y_test, pred_reduced)),
        'rmse': float(np.sqrt(mean_squared_error(y_test, pred_reduced))),
        'mae': float(mean_absolute_error(y_test, pred_reduced)),
    })

comparison_df = pd.DataFrame(model_rows)
comparison_df = comparison_df.sort_values(['rmse', 'mae', 'r2'],␣
 ↪ascending=[True, True, False]).reset_index(drop=True)
comparison_df['rank'] = np.arange(1, len(comparison_df) + 1)
comparison_df = comparison_df[['rank', 'model_name', 'r2', 'rmse', 'mae']]
print(comparison_df)

best_model_name = comparison_df.iloc[0]['model_name']
print('lowest RMSE):', best_model_name)
```

```
   rank                model_name        r2      rmse       mae
0     1     Combined Stepwise OLS  0.726989  6.141604  4.722126
1     2          Original Full OLS  0.724350  6.171214       NaN
2     3  Reduced-VIF Selected OLS  0.720014  6.219570  4.692757
lowest RMSE): Combined Stepwise OLS
```

1.18 15. Final Railway Desert

[49]:
```
df['predicted_target'] = analysis_model.predict(X_analysis)
df['access_gap'] = df[target_col] - df['predicted_target']
df['rail_desert_flag'] = df['access_gap'] < df['access_gap'].quantile(0.10)

# Coefficient based weighted Rail Accessibility Score (RAS)
coefficient_weights = analysis_model.params.drop(labels='const',␣
 ↪errors='ignore')
coefficient_predictors = [c for c in coefficient_weights.index if c in␣
 ↪X_analysis.columns]
coefficient_weights = coefficient_weights.loc[coefficient_predictors]

coefficient_contributions = X_analysis[coefficient_predictors].
 ↪mul(coefficient_weights, axis=1)
df['coefficient_weighted_access_score'] = coefficient_contributions.sum(axis=1)

# Normalization
score_min = df['coefficient_weighted_access_score'].min()
score_max = df['coefficient_weighted_access_score'].max()
if np.isclose(score_max, score_min):
    df['rail_accessibility_score'] = 0.5
else:
    df['rail_accessibility_score'] = (
        (df['coefficient_weighted_access_score'] - score_min) /
        (score_max - score_min)
    )
df['rail_accessibility_score'] = df['rail_accessibility_score'].clip(0, 1)

print('Rail Accessibility Score weighting: selected-model coefficients')
print()
print('Coefficient weights used in the index:')
display(coefficient_weights.to_frame('coefficient_weight'))

df[[
    'district', 'state', 'rail_accessibility_score',
    'coefficient_weighted_access_score', 'predicted_target',
    'access_gap', 'rail_desert_flag'
]].sort_values(['rail_accessibility_score', 'access_gap']).head(25)
```

```
Rail Accessibility Score weighting: selected-model coefficients

Coefficient weights used in the index:
```

	coefficient_weight
rail_density_log	1.028337e+01
sector_primary_sector_constant_prices_in_millio…	-5.192236e-02
sector_primary_sector_current_shares_percent	5.614425e-02

```
Youth Unemployment Rate (%)                                  -6.741481e+00
access_potential_log                                         -5.484530e+00
stop_per_station                                              9.022050e-02
sector_total_constant_prices_in_millions_rs                   5.261330e-02
sector_per_capita_current_prices_1000_in_rs                  -8.935678e-02
sector_econ_available                                        -1.949987e+02
distance_to_nearest_major_hub_log                            -2.454734e+00
ICT Sector Employment (%)                                     3.704923e-01
distance_to_nearest_major_hub_km                              4.674329e-03
SME Employment (%)                                           -5.881351e-01
Tourism Sector Employment (%)                                -1.301489e+00
sector_tertiary_sector_current_prices_millions_…             -5.713988e-07
rail_density                                                  1.199024e-04
train_per_1000_pts                                           -1.347332e-02
Patents per 100,000 Inhabitants                               1.540877e+00
R&D Expenditure (% of GDP)                                   -5.134505e+00
sector_sector_year                                           -4.379531e+00
sector_primary_sector_current_prices_in_million_rs           -1.431661e-04
sector_tertiary_sector_constant_shares_percent                9.082361e+01
sector_secondary_sector_current_shares_percent                5.077391e-01
sector_secondary_sector_constant_prices_million…             -5.254626e-02
Unemployment Rate (%)                                         9.782991e-01
econ_available                                                8.488241e-01
shrug_available                                              -2.546255e+00
station_per_1000_pts                                          1.199024e-01
sector_primary_sector_constant_shares_percent                 9.048529e+01
sector_secondary_sector_constant_shares_percent               9.028872e+01
sector_tertiary_sector_constant_prices_millions…             -5.259508e-02
major_hub_flag                                                8.488241e-01
sector_year_label_2008 (2004)                                 4.859729e+00
sector_year_label_2009 (2004)                                 9.250262e+00
sector_year_label_2010 (2004)                                 1.452108e+01
sector_year_label_2011 (2004)                                 1.953037e+01
sector_year_label_2012 (2004)                                 2.432096e+01
sector_year_label_2013 (2004)                                 2.947106e+01
```

[49]:

	district	state	rail_accessibility_score	\
1122	lahul and spiti	himachal pradesh	0.000000	
1123	lahul and spiti	himachal pradesh	0.029717	
1121	lahul and spiti	himachal pradesh	0.079172	
1389	panna	madhya pradesh	0.094369	
1120	lahul and spiti	himachal pradesh	0.100247	
1117	lahul and spiti	himachal pradesh	0.102212	
1021	kinnaur	himachal pradesh	0.104536	
1408	pathanamthitta	kerala	0.106918	
1394	panna	madhya pradesh	0.108126	
1390	panna	madhya pradesh	0.108577	

1020	kinnaur	himachal pradesh	0.111725
1391	panna	madhya pradesh	0.113163
1395	panna	madhya pradesh	0.113547
1119	lahul and spiti	himachal pradesh	0.113617
1019	kinnaur	himachal pradesh	0.114195
1018	kinnaur	himachal pradesh	0.115593
1392	panna	madhya pradesh	0.122291
1017	kinnaur	himachal pradesh	0.125340
1393	panna	madhya pradesh	0.126679
1744	subarnapur	odisha	0.129232
1423	perambalur	tamil nadu	0.130259
1015	kinnaur	himachal pradesh	0.131213
1118	lahul and spiti	himachal pradesh	0.140699
376	chandigarh	chandigarh	0.143816
1016	kinnaur	himachal pradesh	0.147733

	coefficient_weighted_access_score	predicted_target	access_gap	\
1122	-15.297832	-15.811688	15.811688	
1123	-13.064935	-13.578792	13.578792	
1121	-9.348836	-9.862692	9.862692	
1389	-8.206935	-8.720791	9.720791	
1120	-7.765294	-8.279150	8.279150	
1117	-7.617607	-8.131464	8.131464	
1021	-7.443028	-7.956884	7.956884	
1408	-7.264005	-7.777862	8.777862	
1394	-7.173235	-7.687092	8.687092	
1390	-7.139405	-7.653262	8.653262	
1020	-6.902837	-7.416694	7.416694	
1391	-6.794793	-7.308649	8.308649	
1395	-6.765900	-7.279756	8.279756	
1119	-6.760655	-7.274512	7.274512	
1019	-6.717221	-7.231078	7.231078	
1018	-6.612194	-7.126051	7.126051	
1392	-6.108947	-6.622804	7.622804	
1017	-5.879803	-6.393660	6.393660	
1393	-5.779223	-6.293080	7.293080	
1744	-5.587388	-6.101245	7.101245	
1423	-5.510234	-6.024091	7.024091	
1015	-5.438527	-5.952383	5.952383	
1118	-4.725722	-5.239578	5.239578	
376	-4.491538	-5.005394	6.005394	
1016	-4.197233	-4.711089	4.711089	

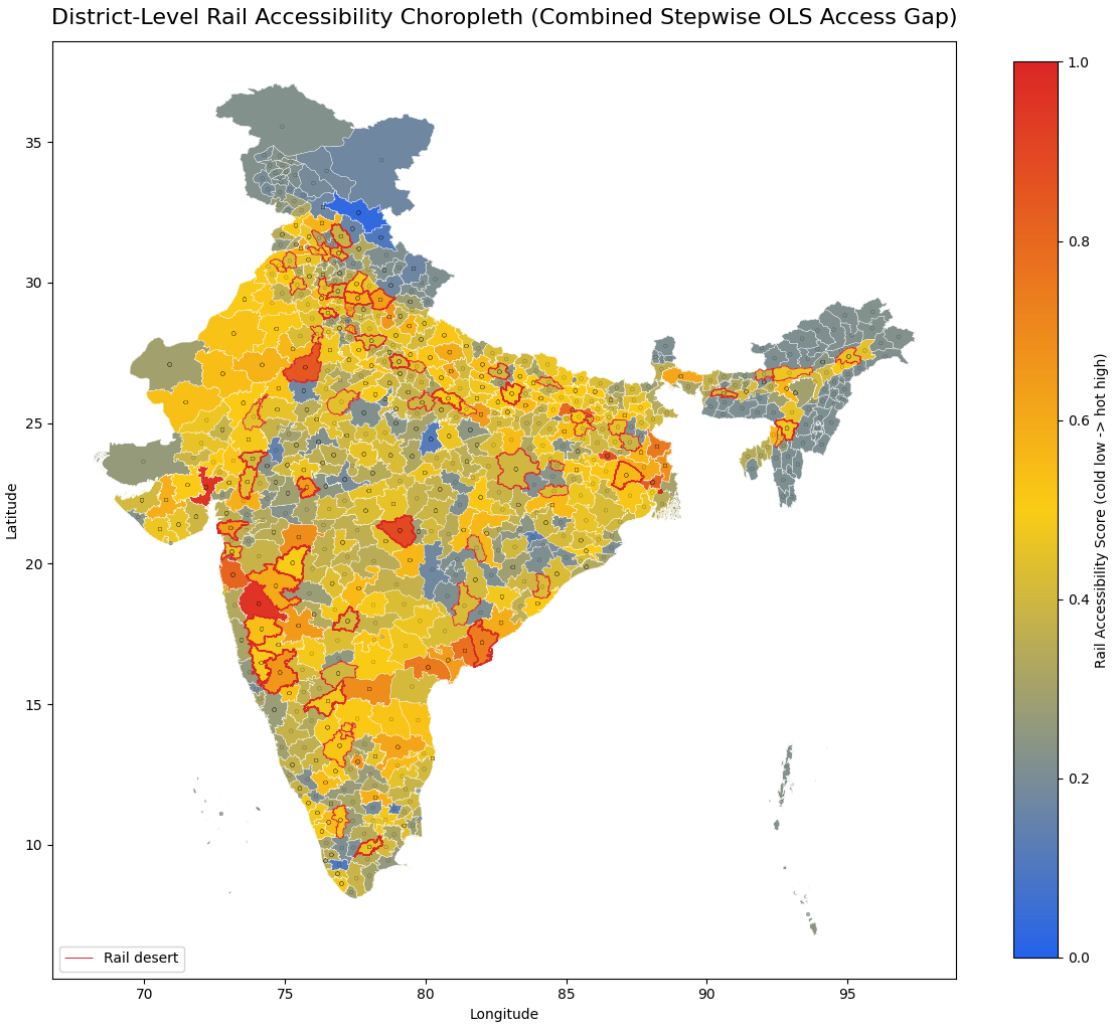
	rail_desert_flag
1122	False
1123	False
1121	False

```
1389            False
1120            False
1117            False
1021            False
1408            False
1394            False
1390            False
1020            False
1391            False
1395            False
1119            False
1019            False
1018            False
1392            False
1017            False
1393            False
1744            False
1423            False
1015            False
1118            False
376             False
1016            False
```

1.19 16. Map the District Results

[50]:
```
# District choropleth map with longitude/latitude centroid overlay
map_cols = [
    'district_key', 'district', 'state',
    'rail_accessibility_score', 'access_gap', 'rail_desert_flag',
    'centroid_lat', 'centroid_lon'
]
map_cols = [c for c in map_cols if c in df.columns]

map_gdf = districts[['district_key', 'geometry']].merge(
    df[map_cols],
    on='district_key',
    how='inner'
).dropna(subset=['rail_accessibility_score', 'centroid_lat', 'centroid_lon'])

if map_gdf.crs is None:
    map_gdf = map_gdf.set_crs(epsg=4326, allow_override=True)
else:
    map_gdf = map_gdf.to_crs(epsg=4326)

fig, ax = plt.subplots(figsize=(12, 14))

map_gdf.plot(
```

```
    column='rail_accessibility_score',
    cmap=cold_warm_hot_cmap,
    linewidth=0.25,
    edgecolor='white',
    legend=True,
    legend_kwds={
        'label': 'Rail Accessibility Score (cold low -> hot high)',
        'shrink': 0.65
    },
    missing_kwds={
        'color': 'lightgrey',
        'edgecolor': 'white',
        'label': 'Missing data'
    },
    ax=ax
)

centroid_plot = map_gdf.dropna(subset=['centroid_lon', 'centroid_lat'])
ax.scatter(
    centroid_plot['centroid_lon'],
    centroid_plot['centroid_lat'],
    c=centroid_plot['rail_accessibility_score'],
    cmap=cold_warm_hot_cmap,
    s=7,
    linewidths=0.15,
    edgecolors='black',
    alpha=0.75
)

if 'rail_desert_flag' in map_gdf.columns:
    desert_gdf = map_gdf[map_gdf['rail_desert_flag'] == True]
    if not desert_gdf.empty:
        desert_gdf.boundary.plot(ax=ax, color=hot_color, linewidth=0.7,␣
 ↪label='Rail desert')
        ax.legend(loc='lower left', frameon=True)

ax.set_title(f'District-Level Rail Accessibility Choropleth␣
 ↪({analysis_model_name} Access Gap)', fontsize=16, pad=12)
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.set_aspect('equal')
ax.grid(False)
plt.tight_layout()
plt.show()
```


District-Level Rail Accessibility Choropleth (Combined Stepwise OLS Access Gap)
Latitude
Longitude
Rail desert
Rail Accessibility Score (cold low -> hot high)
35
30
25
20
15
10
70
75
80
85
90
95
1.0
0.8
0.6
0.4
0.2
0.0